HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING
Faculty of Advanced Education

FINAL-TERM REPORT
COURSE: EMBEDDED SYSTEM DESIGN

BUILDING DEVICE DRIVERS ON LINUX
PART: PLATFORM DRIVERS WITH DEVICE TREE ON THE RASPBERRY PI ZERO W

(Timer, PWM and DS1307 real-time clock)

Lecturer: Huỳnh Hoàng Hà
Course code: EMSY435664E
Group: 6

Ho Chi Minh City, October 2026

LIST OF GROUP MEMBERS

Group: 6

Topic: Building device drivers on Linux – Part: Platform drivers with Device Tree on the Raspberry Pi Zero W

No.	FULL NAME	STUDENT ID	CONTRIBUTION
1	Võ Trần Đăng Khoa	24119051	100%
2	Phạm Trần Huy Hoàng	24119039	100%
3	Huỳnh Anh Tuấn	24119096	100%

Lecturer's comments:

Form:

……

……

……

……

Content:

……

……

……

Total score:

……

……

……

Date … month … year … ***Lecturer***

……

TABLE OF CONTENTS

LIST OF ABBREVIATIONS.....5

INTRODUCTION.....6

1. Reasons for choosing the topic.....6
2. Research objectives.....7
3. Research subjects and scope.....7
4. Research methods.....7
5. Structure of the report.....8

CONTENT.....8

CHAPTER 1: THEORETICAL BASIS AND KEYWORDS OF THE PLATFORM DRIVER.....9

1.1. Keywords of the build process.....9
1.2. The platform driver in the Linux device model.....11
1.2.1. Bus, device and driver.....11
1.2.2. Life cycle of a platform driver.....12
1.2.3. Comparison with other driver types and advantages.....12
1.3. The Device Tree.....13
1.3.1. DTS, DTB, DTBO and overlays.....13
1.3.2. Key properties and the APIs that read them.....14
1.3.3. Why the Device Tree is required.....14
1.4. Kernel frameworks used by the drivers.....15
1.5. Hardware background.....16
1.5.1. BCM2835 System Timer.....16
1.5.2. BCM2835 PWM.....16
1.5.3. DS1307 and the I2C bus.....17
1.6. How the Makefile, DTS and C file work together.....17
1.6.1. Timeline from source to running driver.....18
1.6.2. From physical pin to driver code (RTC example).....18
1.7. Execution order of the driver code and comparison with a character driver…19
1.7.1. Execution order of the functions in the C file.....19
1.7.2. Platform driver versus character driver.....21

CHAPTER 2: IMPLEMENTATION OF THE PLATFORM DRIVERS ON THE RASPBERRY PI ZERO W.....23

2.1. Timer platform driver (bcm_timer_plat).....24
2.1.1. Hardware connection.....24
2.1.2. Preparing Raspberry Pi OS.....24
2.1.3. Source files and Makefile.....24
2.1.4. Device Tree overlay and driver design.....26
2.1.5. Building and loading.....28

2.1.6. Testing and results....29
2.1.7. Unloading and troubleshooting....31
2.2. PWM platform driver (bcm_pwm_plat)....31
2.2.1. Hardware connection....31
2.2.2. Source files....32
2.2.3. Device Tree overlay and driver design....32
2.2.4. Building and loading....34
2.2.5. Testing and results....34
2.2.6. Unloading and troubleshooting....36
2.3. RTC platform driver (ds1307_plat)....37
2.3.1. Hardware connection....37
2.3.2. Enabling I2C and checking the bus....37
2.3.3. Source files....38
2.3.4. Device Tree overlay and driver design....39
2.3.5. Building and permanent loading....40
2.3.6. Testing and results....41
2.3.7. Unloading and troubleshooting....46
2.4. Evaluation and directions for improvement....46
2.4.1. Results achieved....46
2.4.2. Remaining issues....47
2.4.3. Directions for improvement....47
CONCLUSION....48
REFERENCES....49

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ALT0, ALT5	Alternate functions 0 and 5 of a BCM2835 GPIO pin
BCD	Binary-Coded Decimal
CH	Clock Halt bit of the DS1307 seconds register
DT	Device Tree
DTS / DTB / DTBO	Device Tree Source / Blob / Blob for an overlay
GPIO	General-Purpose Input/Output
I2C	Inter-Integrated Circuit bus
IRQ / ISR	Interrupt Request / Interrupt Service Routine
LED	Light-Emitting Diode
MMIO	Memory-Mapped Input/Output
NTP	Network Time Protocol
NVRAM	Non-Volatile (battery-backed) RAM
OS	Operating System
PWM	Pulse-Width Modulation
RTC	Real-Time Clock
SoC	System on Chip
SQW	Square-Wave output of the DS1307
SSH	Secure Shell
UTC	Coordinated Universal Time

INTRODUCTION

1. Reasons for choosing the topic

Embedded Linux now runs on a large share of embedded products, from routers and industrial gateways to single-board computers such as the Raspberry Pi. In these systems almost every peripheral sits inside a System-on-Chip (SoC) at a fixed address and cannot announce itself the way a USB or PCI device does. The device driver is the layer that lets the operating system use such hardware, so building device drivers is a central skill of the Embedded System Design course. Within the group topic "Building device drivers on Linux", this part studies the **platform driver**, the driver model Linux uses for on-chip peripherals.

Firstly, the topic is significant in theory. The platform driver, together with the Device Tree, is the mechanism through which Linux supports nearly every peripheral of an ARM SoC. Studying it brings together the main ideas of kernel programming: loadable modules, the bus–device–driver model, interrupts, memory-mapped registers, resource management and the kernel frameworks that give user space a standard interface. Writing a platform driver therefore consolidates knowledge that a single isolated example cannot cover.

Secondly, the topic has clear practical value. The Raspberry Pi Zero W is low-cost and widely used in student and hobby projects, yet it has no real-time clock and loses the time whenever it has no network. Driving its System Timer and PWM block directly, and adding a DS1307 real-time clock with a backup cell, solves concrete problems: periodic events with measured latency, LED brightness or motor control, and a clock that survives power loss. The results can be checked with simple, visible hardware.

Thirdly, the topic requires the Device Tree. Separating the hardware description from the driver code is what allows one driver binary to run on different boards or pins. Mistakes in this layer, such as a wrong compatible string, a missing phandle or an overlay that is not applied, are a common reason why a driver loads but never works. Understanding how the Makefile, the Device Tree source and the C file meet at run time is therefore necessary for any embedded Linux developer.

For these reasons, the part "Platform drivers with Device Tree on the Raspberry Pi Zero W" was chosen. It combines the theoretical model of the Linux kernel with a complete, tested implementation on real hardware.

2. Research objectives

The general objective is to understand the Linux platform driver model and the Device Tree, and to apply them by building working drivers for real hardware. The specific objectives are:

1. Summarise the keywords and knowledge used in the build process: kernel modules, the device model, the Device Tree, kernel frameworks and the hardware involved.
2. Write three platform drivers and their Device Tree overlays: a periodic timer on BCM2835 System Timer channel 1, a PWM provider that replaces the stock BCM2835 PWM driver, and a driver for the DS1307 real-time clock on I2C1.
3. Document the installation, compilation, loading and testing process so that it can be repeated, and evaluate the results with measured evidence.

3. Research subjects and scope

The research subjects are the Linux platform driver model, Device Tree overlays, and three peripherals controlled through them: the BCM2835 System Timer, the BCM2835 PWM block and the DS1307 real-time clock. The work is limited to a Raspberry Pi Zero W (BCM2835, ARM1176JZF-S, ARMv6) running 32-bit Raspberry Pi OS with kernel 6.18.50+rpt-rpi-v6. The test hardware consists of two LEDs with 330 Ω resistors and a DS1307 module with a CR2032 cell.

The timer and PWM drivers control SoC blocks, which is the case the platform model is designed for. The DS1307 is an external I2C chip that is normally handled by an i2c_driver; in this work it is written as a platform driver in order to apply the Device Tree and the probe()/remove() model required by the topic. The Linux system timer itself cannot be replaced by a module, so the timer driver uses the free compare channel 1.

4. Research methods

Theoretical study: the kernel documentation on the driver model, the Device Tree and the PWM and RTC frameworks, the BCM2835 ARM Peripherals manual and the DS1307 datasheet [1]–[9] were studied to define the register usage and the driver structure.

Experimental method: the drivers were first developed and checked in the QEMU emulator, then built natively and tested on the real Raspberry Pi Zero W over SSH. Their behaviour was observed through sysfs attributes, ioctl calls on /dev/rtc0, /proc/interrupts and the kernel log, and is documented with terminal screenshots and demonstration videos.

Comparison and evaluation: the platform driver is compared with other driver types that could handle the same hardware, and each measured result is compared with the value expected from the datasheets, such as register contents, interrupt counts and the clock offset after a power loss.

5. Structure of the report

Apart from the introduction, the conclusion and the references, the report has two chapters. Chapter 1 summarises the keywords and the theoretical knowledge behind the platform driver. Chapter 2 presents the practical work as three self-contained guides, one for each driver (timer, PWM and RTC), each covering the hardware connection, the source files, the Device Tree overlay and driver design, building and loading, testing and results, and troubleshooting; it ends with an overall evaluation and directions for improvement. The demonstration videos are presented together with the slides of the project.

CONTENT

CHAPTER 1: THEORETICAL BASIS AND KEYWORDS OF THE PLATFORM DRIVER

1.1. Keywords of the build process

Building a platform driver touches five areas of knowledge: the kernel build system, the device model, the Device Tree, kernel APIs and the hardware itself. Table 1.1 summarises the keywords met in each area and where they appear in this work.

Table 1.1 *Keywords used when building the three platform drivers*

Keyword	Meaning	Use in this project
A. Kernel module and build		
Kernel space / user space	Drivers run in privileged kernel space; programs reach them through system calls and virtual files (/sys, /dev).	Drivers in the kernel, tests from the shell
Loadable kernel module (.ko)	Kernel code inserted and removed at run time without rebooting.	bcm_timer_plat.ko, bcm_pwm_plat.ko, ds1307_plat.ko
insmod / rmmod / modprobe / lsmod / depmod	Load a file / unload / load an installed module by name or alias / list / rebuild the alias database.	Loading, testing remove() and probe()
Kbuild, obj-m, KERNELRELEASE	Kernel build system; obj-m lists the modules; KERNELRELEASE is set when Kbuild reads the Makefile.	Two-pass Makefile
Kernel headers, KDIR, vermagic	Build tree of the running kernel (/lib/modules/$(uname -r)/build); the vermagic string in a .ko must match the kernel.	linux-headers-rpi-v6, native build on the Pi
dmesg, modinfo	Kernel log / module metadata (license, alias, vermagic).	Checking probe messages
B. Device model and platform driver		
Bus, device, driver	The three objects of the device model; the bus matches a device with a driver and calls probe().	Platform bus
Platform bus, platform_device	Pseudo-bus for SoC peripherals that cannot be enumerated; one device per enabled DT node, with its resources (MEM, IRQ).	20003000.edu-timer, 2020c000.pwm, ds1307-plat
platform_driver, module_platform_driver()	Driver structure (probe, remove, of_match_table, dev_groups); macro generating module init/exit.	All three drivers
compatible, of_device_id, MODULE_DEVICE_TABLE(of)	Matching string; table of supported strings; exported as module alias so udev loads the module automatically.	"edu,bcm2835-timer-ch1", "edu,bcm2835-pwm-plat", "edu,ds1307-platform"

Keyword	Meaning	Use in this project
probe() / remove()	Called when a device is bound / unbound (rmmod, unbind, overlay removal).	Acquire / release resources
-EPROBE_DEFER	Probe asks to be retried because a supplier (bus, clock) is not ready yet.	RTC waits for i2c-bcm2835
devres (devm_*)	Managed resources released automatically in reverse order on probe failure or unbind.	iomap, IRQ, GPIO, clock, rtc, pwmchip
C. Device Tree		
Device Tree, DTS / DTB / DTBO	Tree of nodes and properties describing non-discoverable hardware; source / compiled blob / compiled overlay.	bcm2708-rpi-zero-w.dtb + three .dtbo
Overlay: /plugin/, fragment, target, __overlay__	Fragment merged into the base tree at boot (config.txt) or at run time (dtoverlay).	&soc, &pwm, &i2c_arm, target-path "/"
dtc -@, label, phandle (&name)	Device Tree compiler; -@ keeps symbols so &label references resolve on the Pi.	&intc, &gpio, &pwm0_gpio18
reg, ranges	Register range in bus addresses; the parent translates 0x7E00_0000 to ARM physical 0x2000_0000.	0x7e003000, 0x7e20c000
interrupts, clocks, pinctrl-0, *-gpios	Consumer bindings to the interrupt, clock, pin-mux and GPIO providers.	IRQ <1 1>, 50 MHz PWM clock, GPIO18 ALT5, GPIO17
status, __overrides__	"okay" creates the device, "disabled" ignores it; overrides are parameters settable from config.txt.	period_us, led_gpio, addr
D. Kernel APIs and frameworks		
MMIO: ioremap, readl / writel	Map physical registers into kernel virtual memory and access them.	Timer CS/CLO/C1, PWM CTL/RNG/DAT
IRQ, ISR, devm_request_irq, synchronize_irq	Interrupt handler runs in atomic context and must not sleep.	Timer ISR
spinlock / mutex	Spinlock usable in IRQ context; mutex may sleep and is never used in an ISR.	Timer, PWM CTL / RTC I2C transfers
PWM framework: pwm_chip, pwm_ops	Provider API (apply, get_state); creates /sys/class/pwm/pwmchipN.	PWM driver
RTC framework, nvmem	rtc_class_ops, devm_rtc_register_device create /dev/rtcN; nvmem exports battery-backed RAM.	RTC driver, hwclock
I2C core: adapter, client, i2c_transfer	Controller object, device address, message transfer with repeated START.	DS1307 on I2C1

Keyword	Meaning	Use in this project
GPIO descriptor API, sysfs attributes	devm_gpiod_get_optional, gpiod_set_value; DEVICE_ATTR_RW/RO, sysfs_emit, dev_groups.	LED GPIO17; period_us, regs, sqw
LINUX_VERSION_CODE	Compile-time selection between kernel API versions.	.remove returns void ≥ 6.11
E. Hardware		
BCM2835 System Timer	1 MHz counter CLO/CHI, compare registers C0–C3; a match sets CS.Mn and raises an IRQ.	Channel 1 (free)
BCM2835 PWM, ALT5	Mark-space mode: period = RNG ticks, high time = DAT ticks; GPIO18 in ALT5 outputs PWM0.	LED brightness
DS1307	I2C RTC at 0x68: BCD time registers, CH bit, 56-byte NVRAM, square-wave output, 5 V supply, CR2032 backup.	RTC driver

1.2. The platform driver in the Linux device model

1.2.1. Bus, device and driver

The Linux device model is built on three objects: a **bus**, the **devices** on it and the **drivers** that serve them. The bus matches each device with a driver and calls the driver's probe(). USB and PCI devices identify themselves, but SoC peripherals sit at fixed addresses and cannot be enumerated, so Linux places them on a pseudo-bus, the **platform bus** [1], [9]. A platform_device holds one peripheral and its resources (registers, interrupt, clock); a platform_driver holds the code. All three drivers of this work register the same way (Listing 1.1).

Listing 1.1 *Registration of the timer platform driver*

```
static const struct of_device_id bcm_timer_of_match[] = {
    { .compatible = "edu,bcm2835-timer-ch1" },
    { /* sentinel */ }
};
MODULE_DEVICE_TABLE(of, bcm_timer_of_match); /* module alias */

static struct platform_driver bcm_timer_driver = {
    .probe  = bcm_timer_probe,
    .remove = bcm_timer_remove,   /* void return since 6.11 */
    .driver = {
        .name           = "bcm-timer-plat",
        .of_match_table = bcm_timer_of_match,
        .dev_groups     = bcm_timer_groups, /* sysfs files */
    },
};
module_platform_driver(bcm_timer_driver);
```

MODULE_DEVICE_TABLE exports the compatible string as a module alias, so udev loads the installed module automatically; dev_groups makes the driver core create the sysfs files after probe() and remove them before remove().

1.2.2. Life cycle of a platform driver

A platform driver has no main() function. The kernel calls its functions in the following order:

1. **Module init:** insmod or modprobe loads the .ko; module_platform_driver() generates the init function, which calls platform_driver_register().
2. **Matching:** the platform bus compares the compatible string of every device created from the Device Tree with the driver's of_match_table.
3. **probe():** on a match the driver core applies the pin configuration and calls probe(), which reads the resources, maps registers, requests the interrupt and registers with a kernel framework. If a supplier (I2C controller, clock) is not ready, probe() returns -EPROBE_DEFER and is retried later.
4. **Running:** the driver reacts to interrupts and to user requests arriving through sysfs, /dev or a framework (ioctl on /dev/rtc0, writes to /sys/class/pwm).
5. **remove():** called on rmmod, sysfs unbind or overlay removal. Resources obtained with devm_* functions are released automatically in reverse order, also when probe() fails halfway.
6. **Module exit:** platform_driver_unregister() removes the driver from the bus.

1.2.3. Comparison with other driver types and advantages

Table 1.2 compares the platform driver with other driver types that could handle the same hardware.

Table 1.2 *Platform driver compared with other driver types*

Criterion	Platform driver + DT (this work)	Character driver, hard-coded addresses	Bus driver (i2c_driver)	User space (/dev/mem, i2c-dev)
Typical hardware	SoC blocks at fixed addresses: timer, PWM, watchdog, clocks	Any simple device	Chips on a bus: sensor, EEPROM, RTC	Prototyping only
Hardware description	DT node (reg, interrupts, clocks, gpios), separate from code	Constants in the C code	DT child node of the bus controller	Constants in the program
Device–driver binding	Automatic by compatible; probe/remove	None; code runs at module init	Automatic by the I2C core	None

Criterion	Platform driver + DT (this work)	Character driver, hard-coded addresses	Bus driver (i2c_driver)	User space (/dev/mem, i2c-dev)
Resource ownership	Kernel reserves registers, IRQ, pins; conflicts rejected	Manual, unchecked	I2C core claims the address	None; can corrupt kernel drivers
Interrupts	Real ISR	Real ISR	Real ISR	Polling only
Kernel frameworks (RTC, PWM, clock, pinctrl)	Yes	Rarely	Yes	No
Change pin / period / board	Edit the overlay; same .ko	Edit C, rebuild	Edit the DT	Edit the program

The comparison shows four main advantages of the platform driver:

1. **Hardware described, not hard-coded:** addresses, IRQ, clock and pins come from the overlay, so the same .ko works on another pin or board.
2. **Automatic binding and life cycle:** probe()/remove() are called on load, unload, unbind or overlay removal, and deferred probing orders the drivers.
3. **Safe resource management:** resources are requested from the kernel and released by devm_*, even when probe() fails halfway.
4. **Reuse of kernel frameworks:** the RTC, PWM, GPIO, clock and pinctrl subsystems are reused, so standard tools (hwclock, /sys/class/pwm) work without new user code.

1.3. The Device Tree

1.3.1. DTS, DTB, DTBO and overlays

The **Device Tree (DT)** is a tree of nodes and properties that describes hardware the kernel cannot discover [2]. It exists in three forms: the **DTS**, the human-readable source; the **DTB**, the compiled blob of the whole board (bcm2708-rpi-zero-w.dtb on the Pi Zero W); and the **DTBO**, a compiled **overlay** that adds or changes nodes of the base tree.

An overlay starts with /plugin/ and contains fragments; each fragment names a **target** (a label such as &soc, or a target-path such as "/") and an **__overlay__** body that is merged into that node. It is compiled with **dtc -@**, which keeps the symbol table so that labels like &gpio resolve against the board tree on the Pi. The overlay is applied either at boot, when the firmware merges every overlay listed as dtoverlay= in /boot/firmware/config.txt, or at run time with sudo dtoverlay -d . <name>, which applies

it to the live tree through configfs [7]. In both cases the kernel creates one platform_device for each node whose status is "okay", and the platform bus then looks for a matching driver.

1.3.2. Key properties and the APIs that read them

Table 1.3 lists the Device Tree properties used by the three drivers and the kernel functions that read them, so no address or pin number is written in the C code.

Table 1.3 *Device Tree properties used in this work and the kernel APIs that read them*

DT property	Meaning	Kernel API in the driver	Driver
compatible	Matching string between node and driver	of_match_table, MODULE_DEVICE_TABLE(of, ...)	All
reg, ranges	Register range in bus addresses; the parent translates 0x7E00_0000 to 0x2000_0000	platform_get_resource() + devm_ioremap(); devm_platform_ioremap_resou rce()	Timer; PWM
interrupts	Interrupt line at the interrupt controller	platform_get_irq()	Timer
clocks, assigned-clock-rates	Clock supplier and requested rate	devm_clk_get_enabled(), clk_get_rate()	PWM
pinctrl-0	Pin function (ALT mode) to apply	Applied by the driver core before probe()	PWM
*-gpios	GPIO line used by the device	devm_gpiod_get_optional(dev, "led", ...)	Timer
phandle (&label)	Reference to another node	of_parse_phandle() + of_get_i2c_adapter_by_node()	RTC
status	"okay" creates the device, "disabled" ignores it	Checked by the kernel when creating devices	All
__overrides__	Parameters settable from config.txt	Resolved by the firmware / dtoverlay	Timer, RTC
edu,period-us, i2c-address	Driver-specific values	of_property_read_u32()	Timer, RTC

1.3.3. Why the Device Tree is required

For a platform driver the Device Tree is not optional, for four reasons:

1. **It creates the device.** Without a node whose compatible matches, the module loads but probe() is never called.
2. **It supplies the resources.** Register range, interrupt, clock, pins and GPIO are read from the node; no address is written in the C code.

3. **It links the driver to its suppliers.** Phandles (&intc, &gpio, &i2c_arm, clocks) connect the device to its providers, and the kernel defers the probe until they are ready.
4. **It carries the configuration.** Interrupt period, LED pin and I2C address are DT properties that can be changed from config.txt through __overrides__, without recompiling.

The .dtbo (built by dtc) and the .ko (built by Kbuild) are separate files that must be deployed together; the compatible string is the contract between them.

1.4. Kernel frameworks used by the drivers

A platform driver rarely talks to user space by itself; it plugs into existing kernel subsystems, which provide the user interface and the safety rules. Table 1.4 lists the mechanisms the three drivers rely on [3], [4], [9].

Table 1.4 *Kernel mechanisms and frameworks used by the drivers*

Mechanism	Main API	Rule / effect	Used by
Memory-mapped I/O	devm_ioremap(), readl(), writel()	Registers are accessed through a kernel virtual address, never through the physical address directly	Timer, PWM
Interrupts	devm_request_irq(), synchronize_irq()	The ISR runs in atomic context: it must be short and must not sleep	Timer
Locking	spin_lock_irqsave(), mutex_lock()	Spinlock where an ISR shares data; mutex around I2C transfers, which may sleep	Timer, PWM / RTC
PWM framework	pwm_chip, pwm_ops.apply / get_state, devm_pwmchip_add()	Creates /sys/class/pwm/pwmchipN; the framework validates and passes a requested state	PWM
RTC framework	rtc_class_ops, devm_rtc_register_device()	Creates /dev/rtcN and /sys/class/rtc/rtcN; hwclock and hctosys work unchanged	RTC
nvmem	devm_rtc_nvmem_register()	Exports the 56-byte battery-backed RAM as a file	RTC
I2C core	of_get_i2c_adapter_by_node(), devm_i2c_new_dummy_device(), i2c_transfer()	Claims the address (UU in i2cdetect) and performs combined write/read transfers	RTC
GPIO descriptors	devm_gpiod_get_optional(), gpiod_set_value()	GPIO taken from the DT; a GPIO that can sleep is rejected for use in an ISR	Timer
sysfs attributes	DEVICE_ATTR_RW/RO, sysfs_emit(), dev_groups	Files created after probe() and removed before remove()	All

Mechanism	Main API	Rule / effect	Used by
Managed resources	devm_* family	Released automatically in reverse order on unbind or probe failure	All

1.5. Hardware background

1.5.1. BCM2835 System Timer

The System Timer at bus address 0x7E003000 is a 64-bit free-running counter clocked at **1 MHz** (CLO holds the low 32 bits, CHI the high 32 bits), so one count is 1 µs. Four compare registers C0–C3 hold deadlines: when CLO equals Cn, bit Mn of the control/status register CS is set and an interrupt is raised. CS is **write-1-to-clear**, so a driver clears only its own bit. C0 and C2 are used by the GPU firmware and C3 by the Linux system timer, which leaves **C1** free for this work (Table 1.5) [5].

Table 1.5 *System Timer registers (offset from 0x7E003000)*

Offset	Register	Function
0x00	CS	Match flags M0–M3; write 1 to clear
0x04 / 0x08	CLO / CHI	Counter, low / high 32 bits (1 µs per count)
0x0C–0x18	C0–C3	Compare registers; C1 is used by bcm_timer_plat

1.5.2. BCM2835 PWM

The PWM block at 0x7E20C000 has two channels driven by a **50 MHz** PWM clock (20 ns per tick). In **mark-space mode** (MSEN = 1) a period lasts RNGn ticks and the output is high for DATn ticks, so the duty cycle is DATn / RNGn. The control register CTL holds the bits of both channels: PWEN (enable), POLA (invert polarity) and MSEN. On the Pi Zero W, GPIO18 outputs channel 1 (PWM0) when it is switched to alternate function **ALT5** (Table 1.6) [5].

Table 1.6 *PWM registers (offset from 0x7E20C000)*

Offset	Register	Function
0x00	CTL	PWEN1 (bit 0), POLA1 (bit 4), MSEN1 (bit 7); channel 2 bits from bit 8
0x04	STA	Status: FIFO state, channel active flags
0x10 / 0x14	RNG1 / DAT1	Period and high time of channel 1, in ticks
0x20 / 0x24	RNG2 / DAT2	Period and high time of channel 2, in ticks

1.5.3. DS1307 and the I2C bus

The DS1307 is a real-time clock on the I2C bus at address **0x68**, at most 100 kHz. It counts seconds to years with leap-year correction and keeps every value in **BCD**. Bit **CH** (clock halt) of the seconds register stops the oscillator; it is set on a new chip or after the battery has run out, and must be cleared by writing the time. The chip needs a 4.5–5.5 V supply; when VCC falls below 1.25 × VBAT it switches to the CR2032 cell, keeps counting and blocks I2C access. Its register map, taken from the datasheet [6], is given in Table 1.7.

Table 1.7 *DS1307 register map*

Address	Content	Note
0x00	Seconds	Bit 7 = CH (oscillator stopped)
0x01	Minutes	BCD
0x02	Hours	Bit 6 selects 12/24-hour mode
0x03	Day of week	1–7
0x04 / 0x05 / 0x06	Date / month / year	BCD, year 00–99
0x07	Control	SQWE (bit 4) and RS1:0 select the square-wave output
0x08–0x3F	NVRAM	56 bytes, battery-backed

The register pointer auto-increments, so all seven time registers are read in one transfer: a write of the start address, a repeated START, then seven reads. The chip copies the time into a buffer at START, so the bytes form one consistent instant.

1.6. How the Makefile, DTS and C file work together

Each driver consists of three source files with separate jobs: the Makefile only builds, the DTS only describes hardware, and the C file only contains code that the kernel calls (Table 1.8). They meet at run time through the compatible string.

Table 1.8 *Role of each source file*

File	Processed by	Output	Used when	Role in the kernel
Makefile	make + Kbuild	.ko (and .dtbo, rtc_test)	Build and install time only	Compiles the C file with the running kernel's flags; copies outputs to /lib/modules and /boot/firmware/overlays
*-overlay.dts	dtc -@	.dtbo	Boot (config.txt) or dtoverlay	Adds or changes DT nodes; the kernel turns each node into a platform_device

File	Processed by	Output	Used when	Role in the kernel
*.c	gcc via Kbuild	.ko	insmod / modprobe and while running	Registers the platform_driver; probe(), callbacks and remove() are called by the kernel

1.6.1. Timeline from source to running driver

1. **Build:** make compiles the .c into a .ko and dtc compiles the .dts into a .dtbo.
2. **Install:** make install copies the .ko to /lib/modules/$(uname -r)/extra (depmod records its alias) and the .dtbo to /boot/firmware/overlays.
3. **Boot or dtoverlay:** the overlay is merged into the live Device Tree; the kernel creates the platform_device with its resources.
4. **Load:** udev (from the alias) or insmod loads the .ko; the init function registers the platform_driver.
5. **Probe:** the platform bus matches compatible strings and calls probe().
6. **Run:** user requests through sysfs, /dev or a framework reach the driver callbacks; interrupts reach the ISR.
7. **Remove:** rmmod, unbind or dtoverlay -r calls remove(); devm resources are released.

1.6.2. From physical pin to driver code (RTC example)

The overlay does not name the I2C pins itself. They are already described in the board Device Tree, and the overlay only enables that controller and points to it. Table 1.9 follows one read of the time from the wires to the C code.

Table 1.9 *Chain linking the DS1307 wiring to ds1307_plat*

Step	Layer	What connects it
1	Wiring	DS1307 SDA → header pin 3 (GPIO2), SCL → pin 5 (GPIO3)
2	Board DT (bcm2708-rpi-zero-w.dtb)	Pin group i2c1: brcm,pins = <2 3>, brcm,function = <4> (ALT0)
3	Board DT	Controller node i2c@7e804000, label **i2c_arm**, uses that pin group
4	Our overlay, fragment@0	target = <&i2c_arm>: status = "okay", clock-frequency = 100 kHz
5	Stock driver i2c-bcm2835	Binds the controller and registers adapter i2c-1
6	Our overlay, fragment@1	Node /ds1307-plat: compatible, i2c-bus = <&i2c_arm>, i2c-address = <0x68>

Step	Layer	What connects it
7	Platform bus	compatible "edu,ds1307-platform" matches the driver; ds_probe() is called
8	ds_probe()	of_parse_phandle("i2c-bus") → adapter i2c-1 → dummy client at 0x68 → /dev/rtc0
9	User space	hwclock or rtc_test calls ioctl(RTC_RD_TIME) on /dev/rtc0
10	RTC core → ds_read_time()	i2c_transfer() reads registers 0x00–0x06 and converts BCD to struct rtc_time

The timer and PWM drivers follow the same pattern with different links: the timer node supplies reg, interrupts and led-gpios (GPIO17); the PWM overlay applies the pin group pwm0_gpio18 (GPIO18, ALT5) through pinctrl-0.

1.7. Execution order of the driver code and comparison with a character driver

1.7.1. Execution order of the functions in the C file

A driver module has no main() function, so its C file is not executed from top to bottom like an ordinary program. The kernel calls its functions in response to events, in three phases: **load**, **run** and **unload**. Listing 1.2 shows the general flow, which is the same for the three drivers of this work.

Listing 1.2 *General call flow of a platform driver module*

```
insmod / modprobe
 +- module init (generated by module_platform_driver())
     +- platform_driver_register()
         +- platform bus matches "compatible" with a DT node
             +- probe()            <- first function written by us
                 +- driver core creates the dev_groups sysfs files
... run: callbacks on interrupts or user requests ...
rmmod  (or sysfs unbind, dtoverlay -r)
 +- driver core removes the sysfs files
     +- remove()
         +- devm resources released in REVERSE order
             +- module exit -> platform_driver_unregister()
```

Table 1.10 gives the actual order of the functions in each driver. probe() always runs first and remove() always runs last; the functions in between have no fixed order, because they run whenever an interrupt occurs or a user reads or writes an interface.

Table 1.10 *Order in which the kernel calls the functions of the three drivers*

Phase	Functions called, in order
Timer driver (bcm_timer_plat.c)	
Load: probe()	bcm_timer_probe(): devm_kzalloc() → platform_get_resource() + devm_ioremap() → platform_get_irq() → devm_gpiod_get_optional("led") → of_property_read_u32("edu,period-us") → devm_request_irq(bcm_timer_isr) → bcm_timer_set_enabled(true) → st_arm(), which writes the first deadline C1 = CLO + period
Run: hardware	bcm_timer_isr() at every match of C1: clear M1, toggle the LED, write the next deadline
Run: user	enable_store() → bcm_timer_set_enabled(); period_us_store(); irq_count_show(), missed_show(), max_latency_us_show(), counter_show()
Unload: remove()	bcm_timer_remove() → bcm_timer_set_enabled(false) → synchronize_irq(); then devm: free IRQ → release GPIO → iounmap → kfree
PWM driver (bcm_pwm_plat.c)	
Load: probe()	bcm_pwm_probe(): devm_pwmchip_alloc() with ops = bcm_pwm_ops → devm_platform_ioremap_resource() → devm_clk_get_enabled() + clk_get_rate() → devm_add_action_or_reset(bcm_pwm_hw_off) → devm_pwmchip_add(), which creates /sys/class/pwm/pwmchip0
Run: hardware	– (the PWM block runs on its own once configured)
Run: user	Writes to period, duty_cycle, polarity or enable → PWM framework → bcm_pwm_apply() → ns_to_ticks(), ctl_update(); export → bcm_pwm_get_state(); regs_show(), clock_hz_show()
Unload: remove()	bcm_pwm_remove() (log only); then devm: remove pwmchip0 → bcm_pwm_hw_off() writes CTL = 0 → disable clock → iounmap
RTC driver (ds1307_plat.c)	
Load: probe()	ds_probe(): devm_kzalloc() + mutex_init() → of_parse_phandle("i2c-bus") + of_get_i2c_adapter_by_node() (returns -EPROBE_DEFER if I2C1 is not ready) → devm_i2c_new_dummy_device(0x68) → ds_read_regs(): chip present, CH bit → devm_rtc_allocate_device() + rtc_add_group() → devm_rtc_register_device(), which creates /dev/rtc0 → devm_rtc_nvmem_register()
Run: hardware	At boot the RTC core calls ds_read_time() once to set the system clock (hctosys)
Run: user	ioctl(RTC_RD_TIME) → ds_read_time() → ds_read_regs() → bcd2bin(); ioctl(RTC_SET_TIME) → ds_set_time() → ds_write_regs() with CH = 0; ds_nvram_read() / ds_nvram_write(); sqw_show() / sqw_store(); regs_show()
Unload: remove()	ds_remove() (log only); then devm: unregister NVRAM → remove /dev/rtc0 → release address 0x68 → ds_put_adapter() → kfree

Because bcm_pwm_hw_off() is registered before devm_pwmchip_add(), devres runs it after the PWM chip has been removed, so no user can switch the output on again

once it has been turned off. The same reverse-order rule releases the RTC resources from the last one obtained (NVRAM) to the first (the I2C adapter).

1.7.2. Platform driver versus character driver

Both a character driver and a platform driver have a module init function, but the two init functions do very different work. In a basic character driver, init does everything and assumes that the hardware is present (Listing 1.3).

Listing 1.3 *Init function of a basic character driver (hard-coded hardware)*

```
static int __init mydrv_init(void)
{
    base = ioremap(0x20003000, 0x1c);     /* address in code */
    request_irq(97, my_isr, 0, "mydrv", NULL); /* IRQ in code */
    alloc_chrdev_region(&devno, 0, 1, "mydrv");
    cdev_init(&cdev, &fops);
    cdev_add(&cdev, devno, 1);            /* creates /dev/mydrv */
    return 0;
}
module_init(mydrv_init);
```

As soon as insmod runs, this code touches the registers; the kernel does not check whether the hardware exists, whether another driver already uses it, or whether the address is correct. In a platform driver, the init function generated by module_platform_driver() only registers the driver with the bus (Listing 1.4).

Listing 1.4 *Init function generated by module_platform_driver()*

```
static int __init bcm_timer_driver_init(void)
{
    /* only registers the driver; no hardware access */
    return platform_driver_register(&bcm_timer_driver);
}
```

All hardware work is moved into probe(), which the bus calls only when the Device Tree contains a matching node, passing the resources read from that node. This was confirmed on the Pi: loading bcm_timer_plat.ko before the overlay succeeds, but nothing appears in dmesg and the LED stays dark, because no device exists yet and probe() is not called; probe() runs as soon as dtoverlay creates the device. Table 1.11 summarises the differences.

Table 1.11 *Basic character driver compared with a platform driver*

Aspect	Basic character driver	Platform driver
Work done by init	Everything: ioremap, request_irq, create /dev	Only platform_driver_register()
Source of address, IRQ, pins	Constants hard-coded in the C file	Device Tree, read in probe()

Aspect	Basic character driver	Platform driver
When the hardware is touched	Immediately at insmod	Only when the bus finds a matching device and calls probe()
Device absent	Registers are still written (the system may hang)	probe() is not called; nothing happens
Number of devices	Usually written for one device	probe() runs once per matching node; one module serves several devices
Supplier not ready	Must be handled by hand	probe() returns -EPROBE_DEFER and is retried (used by the RTC driver to wait for I2C1)
Detach / re-attach	rmmod of the whole module	unbind/bind or dtoverlay -r calls remove()/probe() while the module stays loaded
Resource release	Written by hand in exit, easy to forget	devm_* releases automatically in reverse order
Change of pin or board	Edit the code and rebuild	Edit the overlay; the same .ko is reused

The two notions are not mutually exclusive. A **character device** defines how user space talks to a driver: a file in /dev with open, read, write and ioctl. A **platform driver** defines how the driver is bound to hardware: the bus, the compatible string, probe() and remove(). A platform driver can therefore create a character device inside probe(), and ds1307_plat does exactly that: /dev/rtc0 is a character device (ls -l /dev/rtc0 shows the type "c"), created by the RTC framework when ds_probe() calls devm_rtc_register_device(). The character driver compared in Table 1.2 is thus the older style that hard-codes the hardware and does all its work in init.

In summary, Chapter 1 has shown that a platform driver is defined by three cooperating parts: the device model that binds code to hardware, the Device Tree that describes the hardware, and the kernel frameworks that expose it to user space. Its code runs event by event, from probe() to remove(), and unlike a basic character driver it never touches hardware that the Device Tree has not described. Chapter 2 applies this knowledge to the three peripherals of the Raspberry Pi Zero W.

CHAPTER 2: IMPLEMENTATION OF THE PLATFORM DRIVERS ON THE RASPBERRY PI ZERO W

This chapter applies the knowledge of Chapter 1 and is organised as three self-contained guides, one for each driver: Section 2.1 the timer driver, Section 2.2 the PWM driver and Section 2.3 the RTC driver. Each guide follows the same steps: hardware connection, source files, Device Tree overlay and driver design, building and loading, testing and results, then unloading and troubleshooting. Section 2.4 evaluates the three drivers together and proposes directions for improvement.

All modules are built natively on the Raspberry Pi Zero W through SSH, so they match the running kernel. Table 2.1 lists the software and hardware shared by the three guides, and Figure 2.1 shows the complete assembled hardware.

Table 2.1 *Software and hardware requirements*

Item	Requirement
Board	Raspberry Pi Zero W with soldered 40-pin header
OS	Raspberry Pi OS 32-bit, up to date (apt full-upgrade)
Kernel	6.18.50+rpt-rpi-v6, with matching linux-headers-rpi-v6
Tools	build-essential (gcc, make), device-tree-compiler (dtc), i2c-tools
Access	SSH from a PC; scp to copy the sources
Components	2 LEDs, 2 × 330 Ω resistors, DS1307 module, CR2032 cell, breadboard, jumper wires

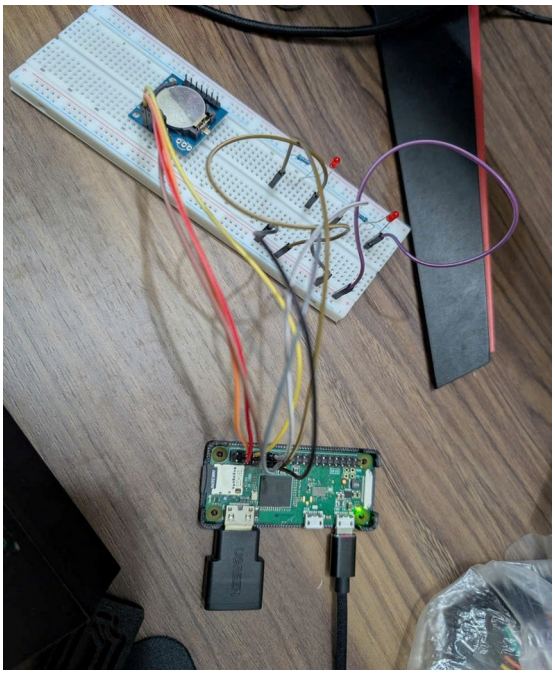


Figure 2.1 *Assembled hardware: Pi Zero W, DS1307 module with CR2032 cell and the two LEDs with their resistors*

In all three guides, sysfs files are written with echo <value> | sudo tee <file> > /dev/null, because in sudo echo <value> > <file> the redirection is done by the unprivileged shell and fails with "Permission denied".

2.1. Timer platform driver (bcm_timer_plat)

The timer driver uses compare channel 1 of the BCM2835 System Timer to generate a periodic interrupt that toggles an LED. Because it is the first driver set up, this guide also contains the one-time preparation of the operating system and the explanation of the Makefile shared by all three drivers.

2.1.1. Hardware connection

The timer LED needs only one GPIO and a current-limiting resistor (Table 2.2). With 3.3 V on GPIO17 and about 2 V across a red LED, the 330 Ω resistor limits the current to roughly 4 mA, well within the GPIO limit.

Table 2.2 *Pin connections of the timer LED*

Component	Pi header pin	Note
LED anode (via 330 Ω)	Pin 11 (GPIO17)	GPIO17 is given in the overlay as led-gpios; it can be changed with the led_gpio parameter
LED cathode	Pin 9 (GND)	Long leg = anode, short leg = cathode

2.1.2. Preparing Raspberry Pi OS

The build tools and the kernel headers are installed once, with the commands of Listing 2.1; the PWM and RTC guides reuse this preparation.

Listing 2.1 *Installing the build tools and kernel headers (done once)*

```
$ sudo apt update && sudo apt full-upgrade -y
$ sudo reboot                       # run the newly installed kernel
$ sudo apt install -y build-essential device-tree-compiler \
      linux-headers-rpi-v6 i2c-tools
$ uname -r                           # 6.18.50+rpt-rpi-v6
$ ls -d /lib/modules/$(uname -r)/build   # headers must exist
```

The headers must match the running kernel, otherwise make fails or insmod reports "Invalid module format", so the Pi is rebooted after a kernel upgrade and the modules are rebuilt [8].

2.1.3. Source files and Makefile

The timer driver lives in its own directory, ~/Timer, with the three files of Table 2.3. They are copied from the PC with the commands of Listing 2.2.

Table 2.3 *Source files of the Timer driver (~/Timer)*

File	Content	Output of make
bcm_timer_plat.c	Platform driver for System Timer channel 1, ISR, sysfs attributes	bcm_timer_plat.ko
bcm-timer-plat-overlay.dts	Node edu-timer@7e003000 in &soc: reg, interrupts, led-gpios, period	bcm-timer-plat.dtbo
Makefile	Kbuild module rule, dtc rule, install/uninstall	–

Listing 2.2 *Copying the timer sources from the PC*

```
$ ssh <user>@<hostname>.local "mkdir -p ~/Timer"
$ scp bcm_timer_plat.c bcm-timer-plat-overlay.dts Makefile \
      <user>@<hostname>.local:~/Timer/
```

If a Makefile was saved by a Windows editor, the command sed -i 's/\r$//' Makefile on the Pi removes the Windows line endings that make cannot parse. The key lines of the Makefile are shown in Listing 2.3; the PWM and RTC Makefiles have the same structure and differ only in file names.

Listing 2.3 *Key lines of the timer Makefile (the others differ only in names)*

```
ifneq ($(KERNELRELEASE),)
obj-m := bcm_timer_plat.o            # pass 2: read by Kbuild
else
KDIR ?= /lib/modules/$(shell uname -r)/build
module:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
%.dtbo: %-overlay.dts
    $(DTC) -@ -I dts -O dtb -o $@ $<
install: all
    install -D -m 644 bcm_timer_plat.ko /lib/modules/$(KVER)/extra/
    depmod -a $(KVER)
    install -m 644 $(DTBOS) $(OVERLAY_DIR)/
endif
```

The Makefile is read twice. In the first pass KERNELRELEASE is empty, so `make -C $(KDIR) M=$(PWD) modules` enters the kernel build tree. Kbuild then re-reads the Makefile with KERNELRELEASE set and compiles obj-m with the kernel's own flags, so the vermagic of the .ko matches the running kernel [9]. dtc -@ keeps the symbol table so labels such as &soc resolve on the Pi. make install copies the .ko to /lib/modules/.../extra, runs depmod and copies the .dtbo to /boot/firmware/overlays.

2.1.4. Device Tree overlay and driver design

The timer overlay adds a new node under &soc (Listing 2.4). This is the first of the three usual ways of describing a platform device; the PWM overlay overrides an existing node and the RTC overlay adds a root node that references a bus.

Listing 2.4 *Key part of bcm-timer-plat-overlay.dts*

```
fragment@0 {
    target = <&soc>;           /* 0x7e... translated by ranges */
    __overlay__ {
        edu_timer: edu-timer@7e003000 {
            compatible = "edu,bcm2835-timer-ch1";
            reg = <0x7e003000 0x1c>;    /* CS, CLO, CHI, C0..C3 */
            interrupt-parent = <&intc>;
            interrupts = <1 1>;          /* GPU IRQ 1 = match 1 */
            led-gpios = <&gpio 17 0>;    /* GPIO17, active high */
            edu,period-us = <500000>;
        };
    };
};
__overrides__ {
    period_us = <&edu_timer>, "edu,period-us:0";
    led_gpio  = <&edu_timer>, "led-gpios:4";
};
```

The node is placed under &soc because it has registers: the soc bus translates 0x7E00_xxxx to ARM physical 0x2000_xxxx, giving the device 20003000.edu-timer. Linux's own timer@7e003000 node is not modified. probe() reads these resources as in Table 1.3. It rejects a GPIO that can sleep, because the LED is toggled in the ISR, and only maps the shared register block with devm_ioremap() without reserving it. After devm_request_irq() it writes the first deadline C1 = CLO + period. The interrupt handler is shown in Listing 2.5.

Listing 2.5 *Interrupt handler of channel 1 (simplified)*

```
static irqreturn_t bcm_timer_isr(int irq, void *data)
{
    struct bcm_timer *t = data;
    u32 now;

    if (!(readl(t->base + ST_CS) & CS_MATCH))
        return IRQ_NONE;
    writel(CS_MATCH, t->base + ST_CS);      /* clear M1 only */

    spin_lock(&t->lock);
    now = readl(t->base + ST_CLO);
    if (t->enabled && (s32)(now - t->next) >= 0) {
        t->max_latency = max(t->max_latency, now - t->next);
        t->irq_count++;
        gpiod_set_value(t->led, t->led_on ^= 1);  /* toggle */
        t->next += t->period_us;                  /* no drift */
        if ((s32)(t->next - now) < 10) {          /* too late */
            t->missed++;
            t->next = now + t->period_us;
        }
        writel(t->next, t->base + ST_C(1));
    }
    spin_unlock(&t->lock);
    return IRQ_HANDLED;
}
```

CS is write-1-to-clear, so writing only M1 never clears the GPU or Linux flags. Each deadline is the previous one plus the period, so the rhythm does not drift with latency, and the latency itself (CLO − deadline) is recorded; signed differences stay correct when the 32-bit CLO wraps, about every 71 minutes. The ISR shares a spinlock with the sysfs functions, and remove() calls synchronize_irq() before devm frees the IRQ. The driver is controlled through the attributes of Table 2.4.

Table 2.4 *sysfs attributes of bcm_timer_plat (directory /sys/bus/platform/drivers/bcm-timer-plat/20003000.edu-timer)*

Attribute	Access	Meaning
enable	rw	1 = run, 0 = stop; writing 1 resets the statistics
period_us	rw	Interrupt period, 100 – 10 000 000 µs
irq_count	r	Interrupts since the last enable
missed	r	Periods skipped because the interrupt came too late
max_latency_us	r	Largest delay from compare match to ISR entry (µs)
counter	r	64-bit System Timer counter (µs since power-on)

2.1.5. Building and loading

The module and the overlay are built with make (Listing 2.6). Kbuild prints the CC, MODPOST and LD steps and dtc builds the .dtbo (Figure 2.2); the path /usr/src/linux-headers-6.18.50+rpt-rpi-v6 confirms that the module is built for the running kernel.

Listing 2.6 *Building the timer driver*

```
$ cd ~/Timer && make
$ ls *.ko *.dtbo        # bcm_timer_plat.ko  bcm-timer-plat.dtbo
$ modinfo ./bcm_timer_plat.ko | grep -e alias -e vermagic
```

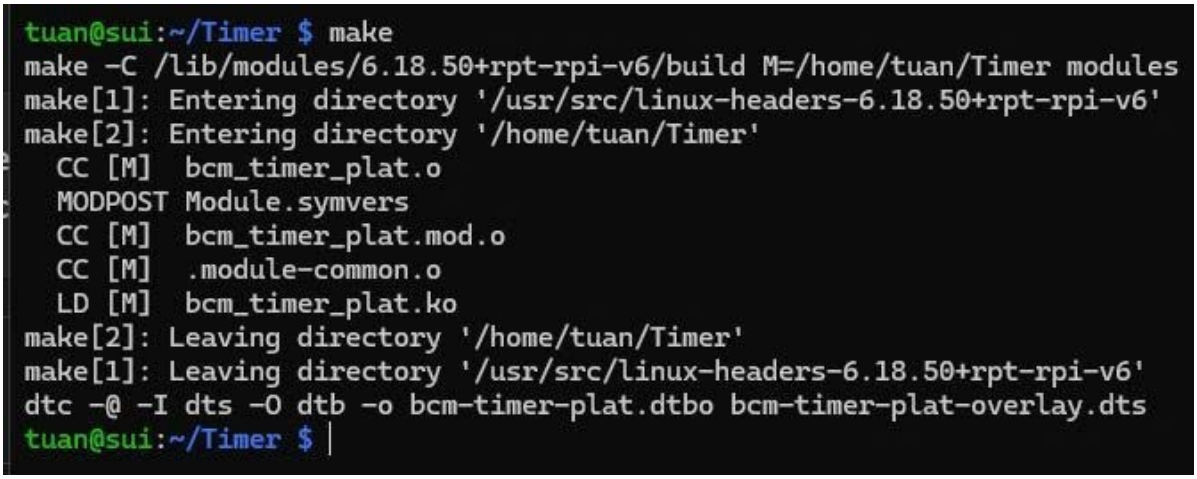


Figure 2.2 *Building bcm_timer_plat.ko and bcm-timer-plat.dtbo*

An overlay and its module can be loaded in two ways (Table 2.5). The same two methods apply to the PWM and RTC drivers; the timer was loaded temporarily for testing.

Table 2.5 *Temporary and permanent loading of a platform driver*

Method	Commands	After reboot	Used for
Temporary	sudo dtoverlay -d . <name>, then sudo insmod ./<module>.ko	Lost	Development, Timer and PWM tests
Permanent	sudo make install, add dtoverlay=<name> to /boot/firmware/config.txt, reboot	Kept; udev runs modprobe from the module alias at every boot	RTC

Listing 2.7 *Loading the timer driver*

```
$ cd ~/Timer
$ sudo dtoverlay -d . bcm-timer-plat     # 1. create the device
$ sudo insmod ./bcm_timer_plat.ko        # 2. register the driver
$ sudo dmesg | grep bcm-timer            # probe() message
# permanent alternative:
$ sudo make install
$ echo dtoverlay=bcm-timer-plat | \
      sudo tee -a /boot/firmware/config.txt
```

The order matters for temporary loading: dtoverlay creates the platform device, insmod registers the driver and the bus calls probe(). The probe message confirms the binding: channel 1, IRQ 25 and the 500 ms period taken from the overlay (Figure 2.3).



Figure 2.3 *Loading the timer overlay and module; probe message*

2.1.6. Testing and results

The tests of Listing 2.8 check the interrupt line, the period, the 1 kHz statistics, the rejection of invalid values and remove()/probe().

Listing 2.8 *Test commands for bcm_timer_plat*

```
$ T=$(ls -d /sys/bus/platform/drivers/bcm-timer-plat/*.edu-timer)
$ grep -e timer /proc/interrupts
$ echo 1 | sudo tee $T/enable > /dev/null
$ sleep 2; cat $T/irq_count                          # about 4
$ echo 100000 | sudo tee $T/period_us > /dev/null  # 5 blinks/s
$ echo 1000 | sudo tee $T/period_us > /dev/null    # 1 kHz
$ echo 1 | sudo tee $T/enable > /dev/null
$ sleep 1; cat $T/irq_count $T/missed $T/max_latency_us
$ echo 50 | sudo tee $T/period_us > /dev/null      # rejected
$ echo 0 | sudo tee $T/enable > /dev/null          # stop
$ sudo rmmod bcm_timer_plat; grep -c edu-timer /proc/interrupts
$ sudo insmod ./bcm_timer_plat.ko
```

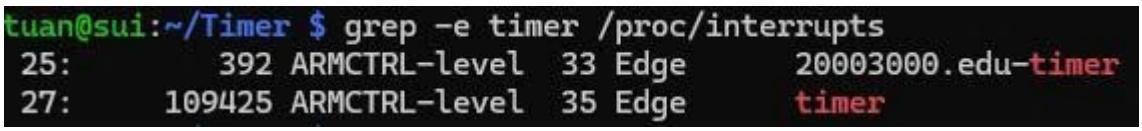


Figure 2.4 *Interrupt lines of System Timer channel 1 (this driver) and channel 3 (Linux)*

Figure 2.4 shows channel 1 (this driver, ARMCTRL 33) and channel 3 (Linux, ARMCTRL 35) interrupting side by side. At 1 kHz the driver handled 1039 interrupts in about 1 s, the extra 39 coming from command overhead, with **no missed period** and a **worst-case latency of 31 µs** (Figure 2.5). After rmmod the edu-timer IRQ line

disappears while /proc/uptime keeps advancing, so the Linux system timer is unaffected (Figure 2.6). Table 2.6 summarises the results.

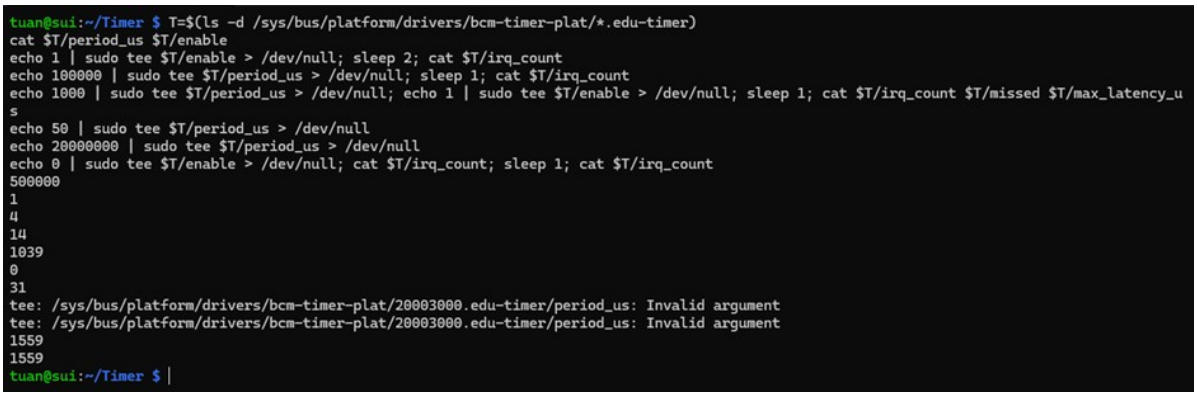


Figure 2.5 *Periodic interrupts, period change, 1 kHz statistics and rejected values of the timer driver*

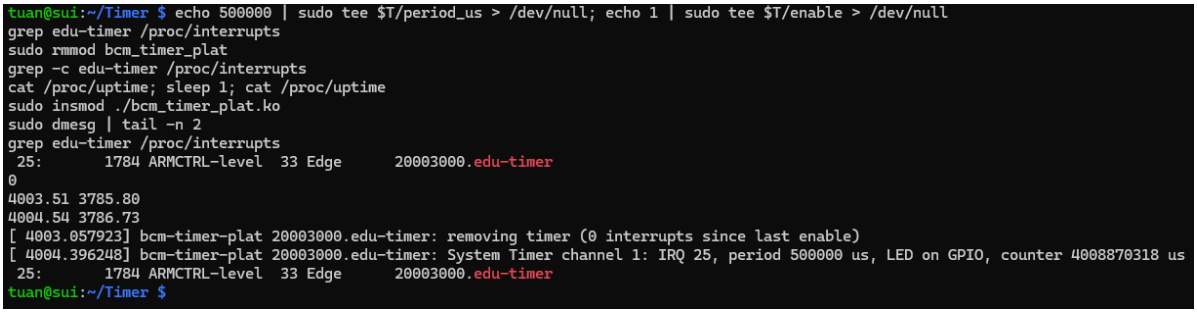


Figure 2.6 *remove() and probe() of the timer driver*

Table 2.6 *Results of the timer driver tests*

Test	Expected behaviour	Result
Two interrupt lines	edu-timer (ARMCTRL 33) and timer (ARMCTRL 35) both counting	Pass (Fig. 2.4)
Period 0.5 s	irq_count ≈ 4 after 2 s; LED blinks once per second	Pass (Fig. 2.5)
Period change	LED blinks 5 times per second at 100 000 µs	Pass (Fig. 2.5)
1 kHz interrupts	irq_count ≈ 1000 per second, no missed period; measured: 1039, missed = 0, max_latency_us = 31	Pass (Fig. 2.5)
Invalid period	50 µs and 20 000 000 µs rejected with "Invalid argument"	Pass (Fig. 2.5)
Stop	irq_count stays constant, LED off	Pass (Fig. 2.5)
remove() / probe()	IRQ line removed, system timer unaffected; LED blinks again after insmod	Pass (Fig. 2.6)

2.1.7. Unloading and troubleshooting

Listing 2.9 *Unloading the timer driver*

```
$ echo 0 | sudo tee $T/enable > /dev/null   # optional: stop first
$ sudo rmmod bcm_timer_plat
$ sudo dtoverlay -r bcm-timer-plat
# permanent install: delete the dtoverlay= line, then
$ sudo make uninstall && make clean
```

rmmod calls bcm_timer_remove(), which stops the interrupt, turns the LED off and waits for a running ISR before devm frees the IRQ. Table 2.7 lists the problems that may appear in this guide and their solutions.

Table 2.7 *Common problems of the timer guide and their solutions*

Symptom	Cause and solution
make: /lib/modules/.../build: No such file or directory	Headers not installed or not matching the kernel: install linux-headers-rpi-v6, full-upgrade, reboot
insmod: Invalid module format	Module built for another kernel: make clean && make after the reboot
No bcm-timer-plat line in dmesg	Overlay not applied: check ls /proc/device-tree/soc/ | grep edu-timer; run dtoverlay before insmod
tee: ...: Permission denied	Write to /sys without root: use echo <value> | sudo tee <file>
cannot get led-gpios / -EBUSY	GPIO17 held by another program: move the LED with dtoverlay=bcm-timer-plat,led_gpio=27 (pin 13)
irq_count increases but the LED stays dark	LED reversed, missing GND or wrong pin (must be pin 11)
irq_count stays at 0	enable is 0: echo 1 | sudo tee $T/enable

2.2. PWM platform driver (bcm_pwm_plat)

The PWM driver drives the BCM2835 PWM block directly and replaces the stock pwm-bcm2835 driver; an LED on GPIO18 shows the duty cycle. The operating system prepared in Section 2.1.2 is reused.

2.2.1. Hardware connection

GPIO18 is the pin that outputs PWM channel 1 (PWM0) in alternate function ALT5 (Table 2.8). /boot/firmware/config.txt must not contain dtoverlay=pwm or dtoverlay=pwm-2chan, otherwise the stock driver takes the PWM block.

Table 2.8 *Pin connections of the PWM LED*

Component	Pi header pin	Note
LED anode (via 330 Ω)	Pin 12 (GPIO18)	ALT5 = PWM0, selected by pinctrl-0 in the overlay

Component	Pi header pin	Note
LED cathode	Pin 14 (GND)	Long leg = anode, short leg = cathode

2.2.2. Source files

The PWM driver lives in ~/PWM with the files of Table 2.9, copied with Listing 2.10. Its Makefile has the same structure as Listing 2.3.

Table 2.9 *Source files of the PWM driver (~/PWM)*

File	Content	Output of make
bcm_pwm_plat.c	Platform driver for the PWM block, pwm_chip provider, regs/clock_hz attributes	bcm_pwm_plat.ko
bcm-pwm-plat-overlay.dts	Overrides &pwm: new compatible, pinctrl GPIO18 ALT5, status okay	bcm-pwm-plat.dtbo
Makefile	Kbuild module rule, dtc rule, install/uninstall	–

Listing 2.10 *Copying the PWM sources from the PC*

```
$ ssh <user>@<hostname>.local "mkdir -p ~/PWM"
$ scp bcm_pwm_plat.c bcm-pwm-plat-overlay.dts Makefile \
      <user>@<hostname>.local:~/PWM/
```

2.2.3. Device Tree overlay and driver design

Instead of adding a node, the PWM overlay overrides the existing &pwm node (Listing 2.11).

Listing 2.11 *Key part of bcm-pwm-plat-overlay.dts*

```
fragment@0 {
    target = <&pwm>;                    /* existing pwm@7e20c000 */
    __overlay__ {
        compatible = "edu,bcm2835-pwm-plat";  /* was brcm,... */
        pinctrl-names = "default";
        pinctrl-0 = <&pwm0_gpio18>;           /* GPIO18 = ALT5 */
        status = "okay";                      /* was "disabled" */
    };
};
```

The new compatible stops the stock pwm-bcm2835 from matching, while reg, clocks and the 50 MHz assigned-clock-rates are inherited from the board tree. The pin state is applied by the driver core just before probe(). The driver is a **PWM provider**: it registers a pwm_chip, and the PWM framework creates /sys/class/pwm/pwmchip0 [3].

Its core is apply() (Listing 2.12), which turns a requested state in nanoseconds into register values.

Listing 2.12 *bcm_pwm_apply(): requested state to register values (simplified)*

```
static int bcm_pwm_apply(struct pwm_chip *chip,
        struct pwm_device *pwm, const struct pwm_state *state)
{
    struct bcm_pwm *pc = to_bcm_pwm(chip);
    unsigned int ch = pwm->hwpwm;              /* channel 0/1 */
    ...
    if (!state->enabled) {
        ctl_update(pc, ch, 0);                 /* clear PWEN */
        return 0;
    }
    range = ns_to_ticks(pc, state->period);    /* 20 ns ticks */
    if (range < RNG_MIN)
        return -EINVAL;                        /* < 40 ns */
    duty = min(ns_to_ticks(pc, state->duty_cycle), range);
    writel(range, pc->base + PWM_RNG(ch));
    writel(duty,  pc->base + PWM_DAT(ch));
    bits = CTL_MSEN | CTL_PWEN;
    if (state->polarity == PWM_POLARITY_INVERSED)
        bits |= CTL_POLA;
    ctl_update(pc, ch, bits);          /* read-modify-write CTL */
    return 0;
}
```

For example, a period of 1 000 000 ns and a duty of 250 000 ns give RNG1 = 50 000, DAT1 = 12 500 and CTL = 0x81 (MSEN + PWEN); inverted polarity adds POLA (CTL = 0x91). CTL is shared by both channels, so ctl_update() runs under a spinlock. probe() maps the registers with devm_platform_ioremap_resource(), enables the clock with devm_clk_get_enabled() and registers a "PWM off" action with devm_add_action_or_reset() before devm_pwmchip_add(), so on removal the output is switched off after the chip is gone. The files used to control the PWM are listed in Table 2.10.

Table 2.10 *Files used to control the PWM ($P = /sys/bus/platform/drivers/bcm-pwm-plat/2020c000.pwm)*

File	Access	Created by	Meaning
$P/regs	r	driver	Dump of CTL, STA, RNG1, DAT1, RNG2, DAT2
$P/clock_hz	r	driver	PWM clock (50 000 000 Hz)
pwmchip0/npwm	r	framework	Number of channels (2)
pwmchip0/export, unexport	w	framework	Create / remove pwmN; unexport before rmmod
pwm0/period	rw	framework	Period in ns (≥ 40 ns), written to RNGn

File	Access	Created by	Meaning
pwm0/duty_cycle	rw	framework	Active time in ns (≤ period), written to DATn
pwm0/polarity	rw	framework	normal / inversed (POLA); only while disabled
pwm0/enable	rw	framework	1 = output on (PWEN + MSEN), 0 = off

2.2.4. Building and loading

Listing 2.13 *Building and loading the PWM driver*

```
$ cd ~/PWM && make                      # bcm_pwm_plat.ko + .dtbo
$ sudo dtoverlay -d . bcm-pwm-plat      # override the &pwm node
$ sudo insmod ./bcm_pwm_plat.ko
$ sudo dmesg | grep bcm-pwm             # probe() message
$ lsmod | grep -i pwm                   # only bcm_pwm_plat
```

The build output (Figure 2.7) has the same steps as for the timer. After loading, the probe message reports two channels and the 50 MHz clock taken from the board Device Tree (Figure 2.8). For a permanent installation the same make install and config.txt method as in Table 2.5 is used, with dtoverlay=bcm-pwm-plat.

```
tuan@sui:~/PWM $ ls
bcm_pwm_plat.c  bcm-pwm-plat-overlay.dts  Makefile
tuan@sui:~/PWM $ make
make -C /lib/modules/6.18.50+rpt-rpi-v6/build M=/home/tuan/PWM modules
make[1]: Entering directory '/usr/src/linux-headers-6.18.50+rpt-rpi-v6'
make[2]: Entering directory '/home/tuan/PWM'
  CC [M]  bcm_pwm_plat.o
  MODPOST Module.symvers
  CC [M]  bcm_pwm_plat.mod.o
  CC [M]  .module-common.o
  LD [M]  bcm_pwm_plat.ko
make[2]: Leaving directory '/home/tuan/PWM'
make[1]: Leaving directory '/usr/src/linux-headers-6.18.50+rpt-rpi-v6'
dtc -@ -I dts -O dtb -o bcm-pwm-plat.dtbo bcm-pwm-plat-overlay.dts
tuan@sui:~/PWM $
```

Figure 2.7 *Building bcm_pwm_plat.ko and bcm-pwm-plat.dtbo*

```
tuan@sui:~/PWM $ sudo dtoverlay -d . bcm-pwm-plat
sudo insmod ./bcm_pwm_plat.ko
tuan@sui:~/PWM $ sudo dmesg | grep bcm-pwm-plat
[ 3762.989282] bcm-pwm-plat 2020c000.pwm: PWM ready: 2 channels, clock 50000000 Hz (20 ns/tick), CTL=0x00000000
tuan@sui:~/PWM $
```

Figure 2.8 *Loading the PWM overlay and module; probe message*

2.2.5. Testing and results

The tests of Listing 2.14 check the driver replacement, the pin function, the register values for several duty cycles, polarity and remove()/probe().

Listing 2.14 *Test commands for bcm_pwm_plat*

```
$ cat /proc/device-tree/soc/pwm@7e20c000/compatible; echo
$ lsmod | grep -i pwm
$ sudo sh -c 'grep "pin 18" \
      /sys/kernel/debug/pinctrl/*/pinmux-pins'
$ P=$(ls -d /sys/bus/platform/drivers/bcm-pwm-plat/*.pwm)
$ C=/sys/class/pwm/pwmchip0
$ echo 0 | sudo tee $C/export > /dev/null
$ echo 1000000 | sudo tee $C/pwm0/period > /dev/null
$ echo 250000 | sudo tee $C/pwm0/duty_cycle > /dev/null
$ echo 1 | sudo tee $C/pwm0/enable > /dev/null; cat $P/regs
$ echo 1000000 | sudo tee $C/pwm0/duty_cycle > /dev/null   # 100%
$ echo 0 | sudo tee $C/pwm0/duty_cycle > /dev/null         # 0%
```

```
tuan@sui:~/PWM $ cat /proc/device-tree/soc/pwm@7e20c000/compatible; echo
lsmod | grep -i pwm
ls /sys/bus/platform/drivers/bcm-pwm-plat/
edu,bcm2835-pwm-plat
bcm_pwm_plat           12288  0
2020c000.pwm  bind  module  uevent  unbind
tuan@sui:~/PWM $
```

Figure 2.9 *PWM node bound to bcm_pwm_plat; the stock pwm_bcm2835 is not loaded*

Figure 2.9 shows that the stock driver is replaced: the node reports edu,bcm2835-pwm-plat and only bcm_pwm_plat is loaded. In Figure 2.10 pin 18 is in ALT5, owned by 2020c000.pwm, and the registers match the requested states (STA = 0x202: FIFO empty, channel 1 active; RNG2 = 32 is the reset value of channel 2). Figure 2.11 shows inverted polarity and remove()/probe(). Table 2.11 summarises the results.

```
tuan@sui:~/PWM $ cd ~/PWM
sudo sh -c 'grep "pin 18" /sys/kernel/debug/pinctrl/*/pinmux-pins'
P=$(ls -d /sys/bus/platform/drivers/bcm-pwm-plat/*.pwm); C=/sys/class/pwm/pwmchip0
echo 0 | sudo tee $C/export > /dev/null
echo 1000000 | sudo tee $C/pwm0/period > /dev/null
echo 250000 | sudo tee $C/pwm0/duty_cycle > /dev/null
echo 1 | sudo tee $C/pwm0/enable > /dev/null; cat $P/regs
echo 1000000 | sudo tee $C/pwm0/duty_cycle > /dev/null; cat $P/regs
echo 0 | sudo tee $C/pwm0/duty_cycle > /dev/null; cat $P/regs
pin 18 (gpio18): device 2020c000.pwm function alt5 group gpio18
CTL=0x00000081 STA=0x00000202 RNG1=50000 DAT1=12500 RNG2=32 DAT2=0
CTL=0x00000081 STA=0x00000202 RNG1=50000 DAT1=50000 RNG2=32 DAT2=0
CTL=0x00000081 STA=0x00000202 RNG1=50000 DAT1=0 RNG2=32 DAT2=0
tuan@sui:~/PWM $
```

Figure 2.10 *GPIO18 in ALT5 and PWM registers for 25 %, 100 % and 0 % duty*

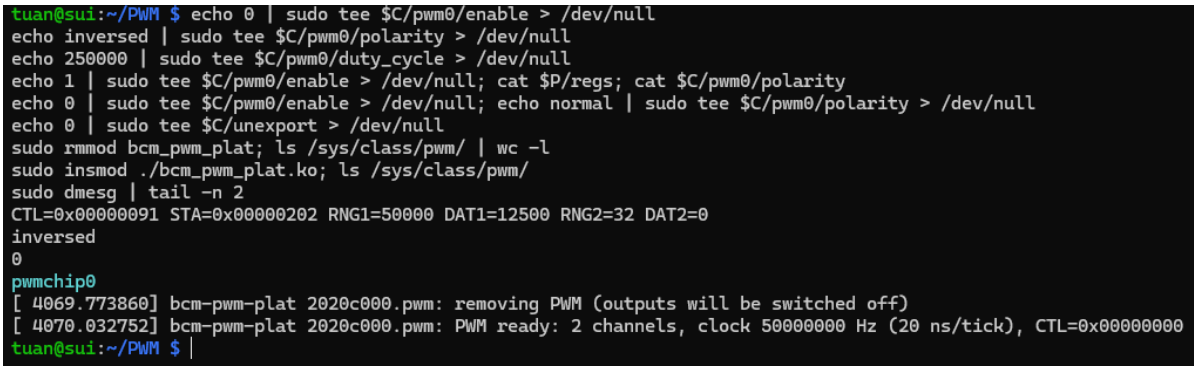


Figure 2.11 *Inverted polarity, remove() and probe() of the PWM driver*

Table 2.11 *Results of the PWM driver tests*

Test	Expected behaviour	Result
Driver replacement	compatible = edu,bcm2835-pwm-plat; only bcm_pwm_plat in lsmod	Pass (Fig. 2.9)
Pin multiplexing	pin 18: function alt5, owned by 2020c000.pwm	Pass (Fig. 2.10)
1 kHz, 25 %	RNG1 = 50000, DAT1 = 12500, CTL = 0x81; LED dim	Pass (Fig. 2.10)
100 % / 0 %	DAT1 = 50000: LED brightest / DAT1 = 0: LED off	Pass (Fig. 2.10)
Breathing loop	LED fades in and out	Pass (demo video)
Inverted polarity	CTL = 0x91; duty 0 brightest, 100 % off	Pass (Fig. 2.11)
Period 20 ns	Rejected: minimum is 2 ticks (40 ns)	Pass
remove() / probe()	pwmchip0 removed and CTL = 0; back after insmod	Pass (Fig. 2.11)

2.2.6. Unloading and troubleshooting

Listing 2.15 *Unloading the PWM driver*

```
$ echo 0 | sudo tee /sys/class/pwm/pwmchip0/unexport > /dev/null
$ sudo rmmod bcm_pwm_plat
$ sudo dtoverlay -r bcm-pwm-plat
```

An exported PWM channel holds a reference to the module, so it is unexported before rmmod. bcm_pwm_remove() lets devm remove pwmchip0 first and then run bcm_pwm_hw_off(), which writes CTL = 0 and switches both outputs off. Table 2.12 lists the problems of this guide.

Table 2.12 *Common problems of the PWM guide and their solutions*

Symptom	Cause and solution
No bcm-pwm-plat line in dmesg	Overlay not applied: cat /proc/device-tree/soc/pwm@7e20c000/compatible must show edu,bcm2835-pwm-plat

Symptom	Cause and solution
pwm_bcm2835 loaded instead of bcm_pwm_plat	dtoverlay=pwm or pwm-2chan still in config.txt: remove it and reboot
cannot get PWM clock	The &pwm node has no clocks property (wrong DTB): check ls /proc/device-tree/soc/pwm@7e20c000/
regs correct but the LED stays dark	LED reversed or wrong pin (must be pin 12); pin 18 must show function alt5
LED blinks instead of dimming	Period too long: use period ≤ 1 000 000 ns (1 ms)
duty_cycle: Invalid argument	duty_cycle larger than period: set period first
rmmod bcm_pwm_plat: module in use	A channel is still exported: unexport it first

2.3. RTC platform driver (ds1307_plat)

The RTC driver handles a DS1307 real-time clock on I2C1, with a CR2032 cell that keeps the time while the Pi is off. Unlike the timer and PWM, it is installed permanently so that the system clock is restored from the RTC at every boot.

2.3.1. Hardware connection

The DS1307 module is connected to the I2C1 pins and to the 5 V supply (Table 2.13).

Table 2.13 *Pin connections of the DS1307 module*

Module pin	Pi header pin	Note
VCC	Pin 4 (5 V)	Must be 5 V: below 1.25 × VBAT the chip blocks I2C access
GND	Pin 6 (GND)	
SDA	Pin 3 (GPIO2, SDA1)	1.8 kΩ pull-up to 3.3 V on the Pi
SCL	Pin 5 (GPIO3, SCL1)	1.8 kΩ pull-up to 3.3 V on the Pi

The DS1307 accepts 3.3 V logic, so no level shifter is needed. The module was checked first with a multimeter: SDA/SCL must not be pulled up to 5 V, because the Pi GPIOs are not 5 V tolerant, and the battery holder must not carry a charging voltage, because the CR2032 is not rechargeable. No other component is required.

2.3.2. Enabling I2C and checking the bus

I2C1 is enabled and the wiring is checked before any driver is loaded (Listing 2.16).

Listing 2.16 *Enabling I2C1 and detecting the DS1307*

```
$ sudo raspi-config nonint do_i2c 0   # adds dtparam=i2c_arm=on
$ sudo reboot
$ ls /dev/i2c-1                       # I2C1 adapter exists
$ sudo i2cdetect -y 1                 # 68 = DS1307, 50 = EEPROM
```

raspi-config prints nothing on success. i2cdetect must show 68 (DS1307) and 50 (the module EEPROM), which confirms the wiring (Figure 2.12); if 68 is missing, the next steps must not be started. config.txt must not contain dtoverlay=i2c-rtc, otherwise the stock rtc-ds1307 takes address 0x68.

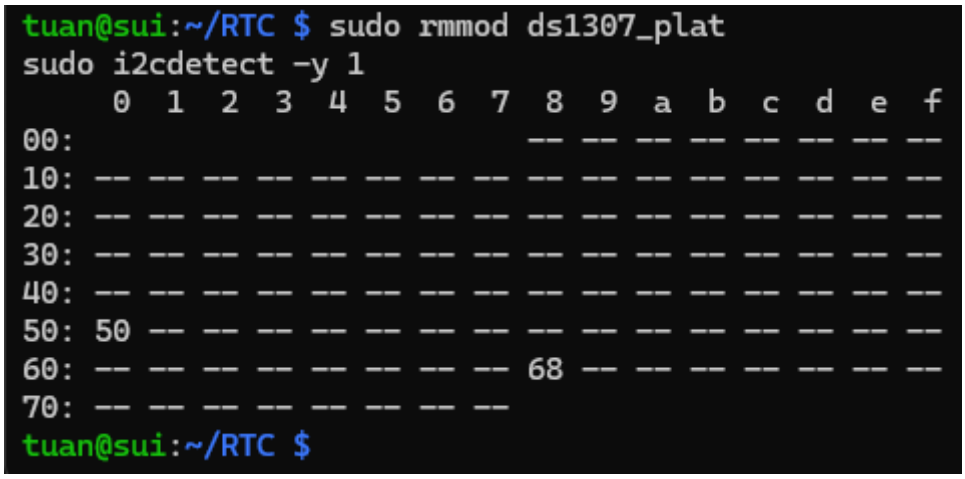


Figure 2.12 *DS1307 (0x68) and the module EEPROM (0x50) detected on I2C1 with the RTC driver unloaded*

2.3.3. Source files

The RTC driver lives in ~/RTC with the four files of Table 2.14, copied with Listing 2.17. Besides the module and the overlay, its Makefile also builds the user-space test program rtc_test.

Table 2.14 *Source files of the RTC driver (~/RTC)*

File	Content	Output of make
ds1307_plat.c	Platform driver for the DS1307 on I2C1: RTC ops, regs/sqw attributes, nvmem	ds1307_plat.ko
ds1307-plat-overlay.dts	Enables &i2c_arm at 100 kHz; root node ds1307-plat with i2c-bus and i2c-address	ds1307-plat.dtbo
rtc_test.c	User-space test tool using ioctl on /dev/rtc0	rtc_test
Makefile	Kbuild module rule, dtc rule, rtc_test rule, install/uninstall	_

Listing 2.17 *Copying the RTC sources from the PC*

```
$ ssh <user>@<hostname>.local "mkdir -p ~/RTC"
$ scp ds1307_plat.c ds1307-plat-overlay.dts rtc_test.c Makefile \
      <user>@<hostname>.local:~/RTC/
```

2.3.4. Device Tree overlay and driver design

The RTC overlay enables I2C1 and adds a root node that points to it (Listing 2.18).

Listing 2.18 *Key part of ds1307-plat-overlay.dts*

```
fragment@0 {
    target = <&i2c_arm>;          /* I2C1: GPIO2 SDA, GPIO3 SCL */
    __overlay__ { status = "okay"; clock-frequency = <100000>; };
};
fragment@1 {
    target-path = "/";
    __overlay__ {
        ds1307_plat: ds1307-plat {
            compatible = "edu,ds1307-platform";
            i2c-bus = <&i2c_arm>;     /* phandle to I2C1 */
            i2c-address = <0x68>;
        };
    };
};
```

The RTC node is at the root, not inside the I2C node, because it is a platform device: it owns no registers and reaches the chip through the I2C1 controller referenced by i2c-bus. The bus runs at 100 kHz, the DS1307 maximum. The main steps of probe() are shown in Listing 2.19.

Listing 2.19 *Key steps of ds_probe() (error checks omitted)*

```
bus_np = of_parse_phandle(np, "i2c-bus", 0);      /* &i2c_arm */
ds->adap = of_get_i2c_adapter_by_node(bus_np);
if (!ds->adap)
    return -EPROBE_DEFER;              /* i2c-bcm2835 not ready */
of_property_read_u32(np, "i2c-address", &addr);   /* 0x68 */
ds->client = devm_i2c_new_dummy_device(dev, ds->adap, addr);
ds_read_regs(ds, DS1307_REG_SECS, regs, 8);    /* chip present? */
if (regs[DS1307_REG_SECS] & DS1307_BIT_CH)
    dev_warn(dev, "oscillator is stopped ...");
ds->rtc = devm_rtc_allocate_device(dev);
ds->rtc->ops = &ds_rtc_ops;                    /* read/set_time */
ds->rtc->range_min = RTC_TIMESTAMP_BEGIN_2000;
ds->rtc->range_max = RTC_TIMESTAMP_END_2099;
rtc_add_group(ds->rtc, &ds_attr_group);          /* regs, sqw */
devm_rtc_register_device(ds->rtc);               /* /dev/rtc0 */
devm_rtc_nvmem_register(ds->rtc, &nvmem_cfg);    /* 56-byte RAM */
```

If the I2C controller is not ready, probe() returns -EPROBE_DEFER and is retried later. devm_i2c_new_dummy_device() claims address 0x68, so i2cdetect shows UU and the stock rtc-ds1307 cannot use the chip at the same time. If every byte reads 0xFF, no chip is answering and probe() fails cleanly. Reading the time is done by ds_read_time() (Listing 2.20).

Listing 2.20 *ds_read_time(): BCD registers to struct rtc_time (simplified)*

```
ds_read_regs(ds, DS1307_REG_SECS, regs, 7);   /* one transfer */
if (regs[0] & DS1307_BIT_CH)
    return -EINVAL;                          /* oscillator stopped */
tm->tm_sec  = bcd2bin(regs[0] & 0x7f);
tm->tm_min  = bcd2bin(regs[1] & 0x7f);
tm->tm_hour = bcd2bin(regs[2] & 0x3f);      /* 24 h (12 h also) */
tm->tm_wday = bcd2bin(regs[3] & 0x07) - 1;
tm->tm_mday = bcd2bin(regs[4] & 0x3f);
tm->tm_mon  = bcd2bin(regs[5] & 0x1f) - 1;
tm->tm_year = bcd2bin(regs[6]) + 100;         /* 00 -> 2000 */
```

With CH = 1 the driver refuses to return a time, and set_time() writes CH = 0 to restart the oscillator. The RTC core [4] rejects dates outside 2000–2099 with -ERANGE, and the I2C transfers, which may sleep, are protected by a mutex. The interfaces the driver provides are listed in Table 2.15.

Table 2.15 *Interfaces provided by ds1307_plat*

Interface	Use
/dev/rtc0	ioctl RTC_RD_TIME / RTC_SET_TIME; used by hwclock and rtc_test
/sys/class/rtc/rtc0/regs	Dump of registers 0x00–0x07
/sys/class/rtc/rtc0/sqw	Square-wave output: off, 1, 4096, 8192, 32768 Hz
/sys/bus/nvmem/devices/ ds1307_nvram0/nvmem	56-byte battery-backed RAM
rtc_test (user program)	read, set "YYYY-MM-DD HH:MM:SS", sync, diff, watch N

2.3.5. Building and permanent loading

Listing 2.21 *Building and permanently installing the RTC driver*

```
$ cd ~/RTC && make            # ds1307_plat.ko, .dtbo, rtc_test
$ sudo make install       # .ko -> /lib/modules, .dtbo -> overlays
$ sudo nano /boot/firmware/config.txt
      dtparam=i2c_arm=on
      dtoverlay=ds1307-plat
$ sudo reboot
$ sudo dmesg | grep ds1307-plat
```

The build output is shown in Figure 2.13 and the two lines added to config.txt in Figure 2.14. After the reboot the firmware merges the overlay, the kernel creates the ds1307-plat device and udev loads the module from its alias, so no insmod is needed.

```
tuan@sui:~/RTC $ ls
ds1307_plat.c  ds1307-plat-overlay.dts  Makefile  rtc_test.c
tuan@sui:~/RTC $ make
make -C /lib/modules/6.18.50+rpt-rpi-v6/build M=/home/tuan/RTC modules
make[1]: Entering directory '/usr/src/linux-headers-6.18.50+rpt-rpi-v6'
make[2]: Entering directory '/home/tuan/RTC'
  CC [M]  ds1307_plat.o
  MODPOST Module.symvers
  CC [M]  ds1307_plat.mod.o
  CC [M]  .module-common.o
  LD [M]  ds1307_plat.ko
make[2]: Leaving directory '/home/tuan/RTC'
make[1]: Leaving directory '/usr/src/linux-headers-6.18.50+rpt-rpi-v6'
dtc -@ -I dts -O dtb -o ds1307-plat.dtbo ds1307-plat-overlay.dts
cc -O2 -Wall -Wextra -o rtc_test rtc_test.c
tuan@sui:~/RTC $
```

Figure 2.13 *Building ds1307_plat.ko, ds1307-plat.dtbo and rtc_test*

```
[all]
dtparam=i2c_arm=on
dtoverlay=ds1307-plat
```

Figure 2.14 *Permanent RTC configuration in /boot/firmware/config.txt*

With a new battery the first probe finds the chip but reads 80 00 00 01 01 01 00 03: CH = 1 (oscillator stopped), so the driver warns and returns no time (Figure 2.15). Setting the time once with sudo ./rtc_test sync (or sudo hwclock -w) clears CH and starts the oscillator.

```
tuan@sui:~/RTC $ dmesg | grep ds1307-plat
[ 5122.486876] ds1307-plat ds1307-plat: DS1307 found at 0x68 on i2c-1, regs: 80 00 00 01 01 01 00 03
[ 5122.486932] ds1307-plat ds1307-plat: oscillator is stopped (new/empty battery?): set the time with 'hwclock -w'
[ 5122.488029] ds1307-plat ds1307-plat: time not valid (oscillator stopped), set it first
[ 5122.488508] ds1307-plat ds1307-plat: registered as rtc0
[ 5122.489612] ds1307-plat ds1307-plat: time not valid (oscillator stopped), set it first
[ 5122.489657] ds1307-plat ds1307-plat: hctosys: unable to read the hardware clock
tuan@sui:~/RTC $
```

Figure 2.15 *First probe of ds1307_plat: chip found at 0x68, oscillator stopped*

2.3.6. Testing and results

The tests of Listing 2.22 check reading and setting the time, rollovers, the valid range, the BCD registers, synchronisation with the system clock, the extra interfaces and the behaviour after a power loss.

Listing 2.22 *Test commands for ds1307_plat (run in ~/RTC)*

```
$ sudo ./rtc_test read; sudo hwclock -r
$ sudo ./rtc_test set "2030-12-31 23:59:58"
$ sudo ./rtc_test watch 4
$ sudo ./rtc_test set "2028-02-28 23:59:59"
$ sleep 2; sudo ./rtc_test read
$ sudo ./rtc_test set "2100-01-01 00:00:00"
$ sudo ./rtc_test set "1999-12-31 23:59:59"
$ sudo ./rtc_test set "2026-10-10 20:35:00"
$ cat /sys/class/rtc/rtc0/regs
$ sudo ./rtc_test sync; sudo ./rtc_test diff
```

rtc_test prints UTC and hwclock prints local time, so Figure 2.16 shows a single instant (13:13:59 UTC = 20:13:59 UTC+7); by Linux convention the RTC stores UTC. Figures 2.17–2.19 show setting the time, the year and leap-year rollovers, and the rejection of out-of-range dates.

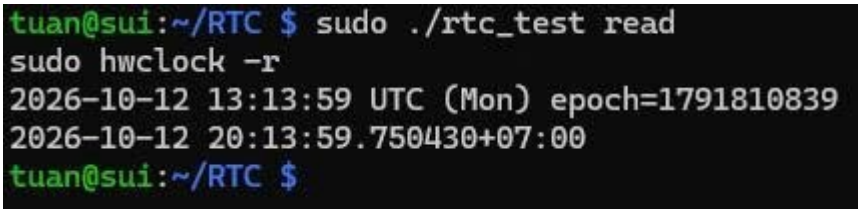


Figure 2.16 *Reading the RTC with rtc_test (UTC) and hwclock (local time)*

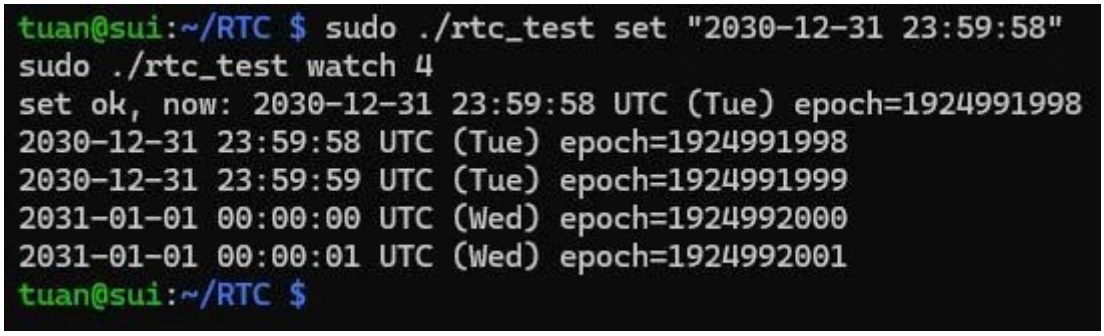


Figure 2.17 *Setting the time and rolling over from 2030 to 2031 (weekday Tue to Wed)*

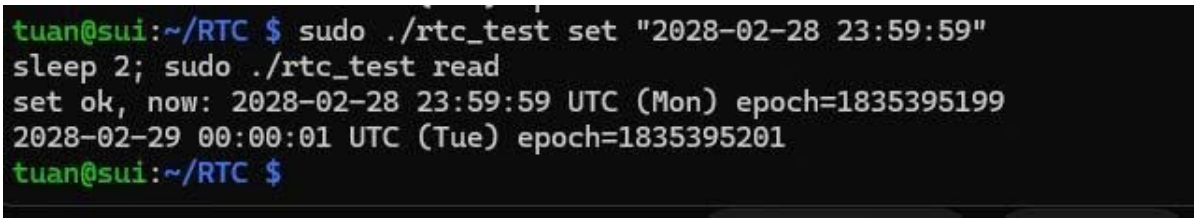


Figure 2.18 *Leap-year rollover from 28 to 29 February 2028*

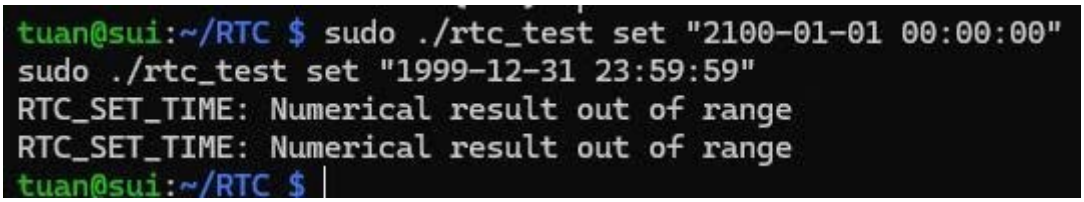


Figure 2.19 *Times outside 2000–2099 rejected by the RTC core (ERANGE)*

The dump 00 35 20 07 10 10 26 00 reads directly as BCD: 20:35:00 (CH = 0, 24-hour mode), weekday 7 (Saturday), 10-10-26, control 00 (Figure 2.20). The −1 s offset in Figure 2.21 comes from the one-second resolution of the DS1307, not from a clock error.

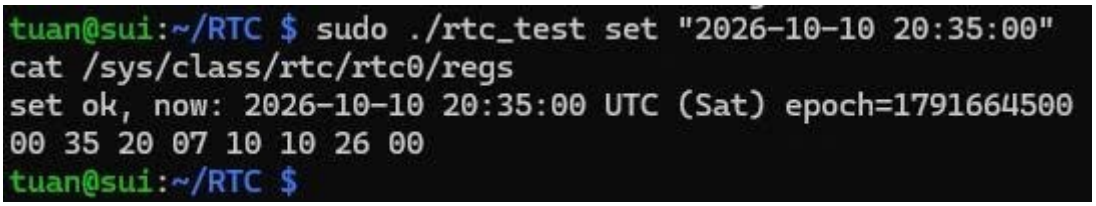


Figure 2.20 *BCD register dump after setting Saturday 2026-10-10 20:35:00*

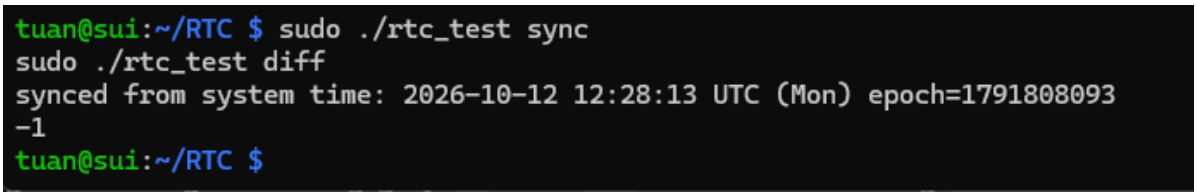


Figure 2.21 *Synchronising the RTC with the system time; offset −1 s*

With the driver bound, address 0x68 shows UU instead of 68 (Figure 2.22 compared with Figure 2.12), confirming devm_i2c_new_dummy_device(). In Figure 2.23 the 1 Hz square wave sets the control byte to 10 (SQWE), the NVRAM string is read back, and after rmmod /dev/rtc0 disappears; after modprobe it returns, the kernel sets the system clock from the RTC and the offset is 0 s.

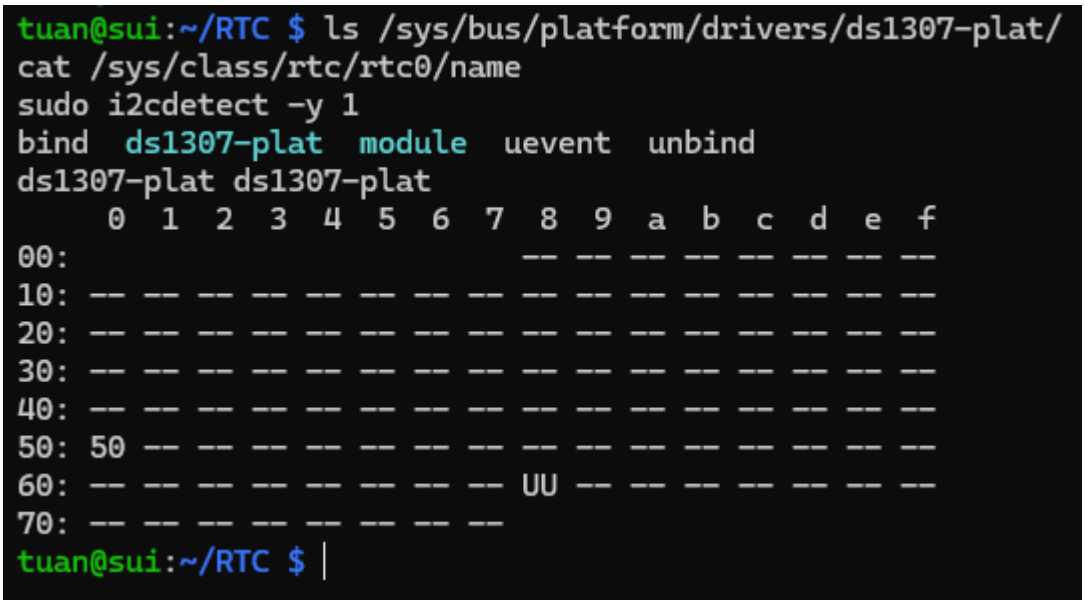


Figure 2.22 *ds1307_plat bound to its device; address 0x68 claimed by the driver (UU)*

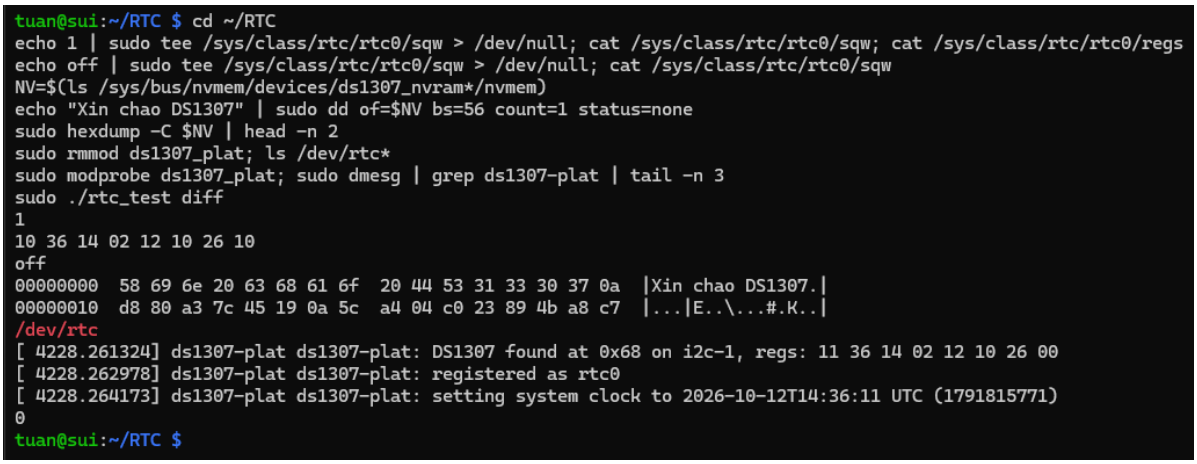


Figure 2.23 *Square-wave output, NVRAM, remove() and probe() of the RTC driver*

Power-loss test. After synchronising, the Pi was shut down with sudo poweroff at 13:15:49 UTC and unplugged for more than 10 minutes. At boot (44 s, before the network was up) the driver read 13:26:27 with CH = 0 and the kernel set the system clock from it; after NTP the offset was 0 s and the NVRAM string was still present (Figure 2.24 and the demo video). Table 2.16 summarises the RTC results.

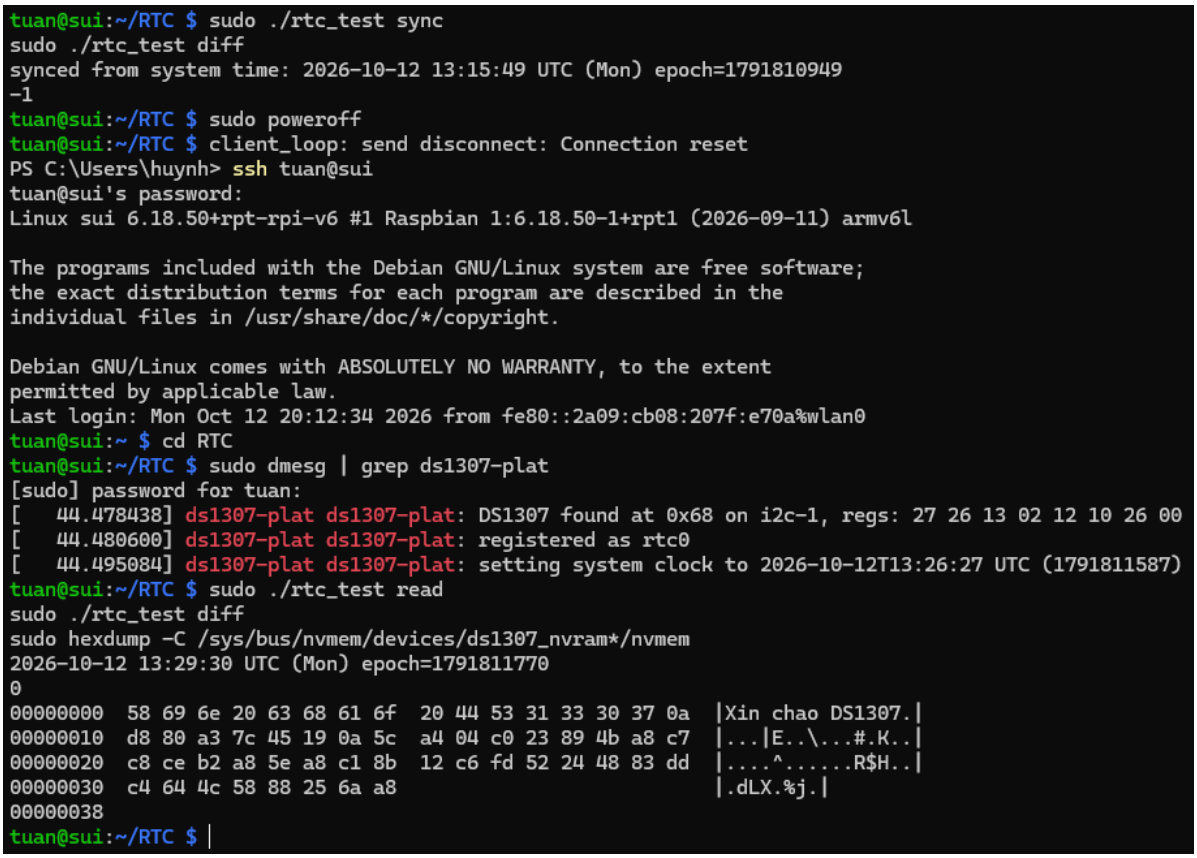


Figure 2.24 *Power-loss test: time restored from the DS1307 at boot, NVRAM content retained*

Table 2.16 *Results of the RTC driver tests*

Test	Expected behaviour	Result
First probe	Chip found at 0x68; CH = 1 warning with a new battery	Pass (Fig. 2.15)
Read	rtc_test (UTC) and hwclock (UTC+7) show the same instant	Pass (Fig. 2.16)
Set and year rollover	2030-12-31 23:59:59 → 2031-01-01 00:00:00	Pass (Fig. 2.17)
Leap year	2028-02-28 23:59:59 + 2 s → 2028-02-29	Pass (Fig. 2.18)
Out of range	Years 2100 and 1999 rejected (ERANGE)	Pass (Fig. 2.19)
BCD registers	00 35 20 07 10 10 26 00 for Sat 2026-10-10 20:35:00	Pass (Fig. 2.20)
Sync with system time	Offset 0 or ±1 s	Pass (Fig. 2.21)
Binding, SQW, NVRAM	UU at 0x68; sqw accepts off/1/4096/8192/32768; 56-byte RAM read/write	Pass (Fig. 2.22, 2.23)
remove() / probe()	/dev/rtc0 removed and re-created; offset still ≈ 0	Pass (Fig. 2.23)
Power loss ≥ 10 min	Time kept by the CR2032; system clock set from the RTC at boot; NVRAM kept	Pass (Fig. 2.24, demo video)

2.3.7. Unloading and troubleshooting

Listing 2.23 *Unloading and uninstalling the RTC driver*

```
$ sudo ./rtc_test sync          # keep the RTC correct first
$ sudo rmmod ds1307_plat
$ sudo modprobe ds1307_plat     # reload (installed module)
# uninstall: delete dtoverlay=ds1307-plat from config.txt, then
$ sudo make uninstall && make clean && sudo reboot
```

rmmod calls ds_remove(); devm then removes the NVRAM and /dev/rtc0 and releases address 0x68, while the DS1307 keeps counting on its battery. The RTC is synchronised before a reload because the kernel sets the system clock from it when /dev/rtc0 is registered again. Table 2.17 lists the problems of this guide.

Table 2.17 *Common problems of the RTC guide and their solutions*

Symptom	Cause and solution
i2cdetect does not show 68	VCC at 3.3 V instead of 5 V, SDA/SCL swapped, loose wire, I2C not enabled (ls /dev/i2c-1)
no DS1307 answering at 0x68 (err -121)	The chip does not acknowledge: check the wiring and the 5 V supply, then reload the driver
cannot claim address 0x68 (-16)	Stock rtc-ds1307 owns the chip: remove dtoverlay=i2c-rtc from config.txt or rmmod rtc_ds1307
No /dev/rtc0 and no error message	probe() deferred: I2C bus not enabled (dtparam=i2c_arm=on missing); see /sys/kernel/debug/devices_deferred
No ds1307-plat line in dmesg	Overlay not applied: check ls /proc/device-tree/ds1307-plat and the .dtbo name in the overlays folder
rtc_test read: Invalid argument	CH = 1, oscillator stopped (new or empty battery): sudo ./rtc_test sync
Time lost after power-off	CR2032 empty or reversed, or the module's charging circuit damaged it

2.4. Evaluation and directions for improvement

2.4.1. Results achieved

All three platform drivers were built natively and passed every planned test (Tables 2.6, 2.11 and 2.16). The timer driver produced periodic interrupts from the free System Timer channel at up to 1 kHz with no missed period and a worst-case latency of 31 µs, without disturbing the Linux system timer. The PWM driver replaced the stock pwm-bcm2835 through an overlay, and duty cycle and polarity were confirmed in the registers and on the LED. The RTC driver made /dev/rtc0 work with standard tools, and the DS1307 kept the time and its NVRAM through a power loss of more than 10

minutes. In every case the hardware description lives only in the overlay, and remove()/probe() release and recreate the device cleanly.

2.4.2. Remaining issues

1. The DS1307 is written as a platform driver to meet the topic; for an I2C chip, an i2c_driver (as in the stock rtc-ds1307) is the conventional design.
2. Interrupt latency was measured only on an idle Pi; it was not measured under CPU or I/O load.
3. Only PWM channel 1 is routed to a pin (GPIO18); channel 2 is registered but not tested on hardware.
4. The drivers have no power-management callbacks (suspend/resume).

2.4.3. Directions for improvement

To address these issues, the following improvements are proposed:

1. Add suspend/resume callbacks to the three drivers and measure the timer latency under CPU and I/O load.
2. Use the PWM driver to control a servo or a motor, routing channel 2 to GPIO19 and testing both channels together.
3. Port the RTC to an i2c_driver for comparison, or replace the DS1307 with a DS3231, which adds temperature compensation and alarms that can wake the system.

CONCLUSION

This part of the report has studied and applied the Linux platform driver model with the Device Tree on the Raspberry Pi Zero W. On the theoretical side, it summarised the keywords of the build process and explained how a platform driver is matched, probed and removed, why the Device Tree is required, which kernel frameworks a driver reuses, and how the Makefile, the overlay and the C file meet at run time. The comparison with character drivers, bus drivers and user-space access showed that the platform driver is the most suitable model for on-chip peripherals, because it separates the hardware description from the code and manages resources safely.

On the practical side, three platform drivers were written, built natively and tested: a periodic timer on System Timer channel 1, a PWM provider that replaces the stock driver, and a driver for the DS1307 real-time clock. All planned tests passed. The timer reached 1 kHz with no missed period and a worst-case latency of 31 µs, the PWM output matched the requested duty cycle and polarity in the registers and on the LED, and the RTC kept the time and its NVRAM through a power loss of more than ten minutes. The installation and compilation process was documented step by step, together with the common problems and their solutions.

The work still has limitations: the DS1307 is handled by a platform driver rather than the conventional i2c_driver, latency was measured only on an idle system, PWM channel 2 was not tested on a pin, and power management is not implemented. These points define the next steps: adding suspend/resume support, measuring latency under load, driving a servo or motor with both PWM channels, and moving to an RTC with alarms such as the DS3231. Through this part, the group has gained a practical understanding of how Linux binds drivers to hardware on embedded systems, which can be applied directly to other boards and peripherals.

REFERENCES

[1] The Linux Kernel Documentation, "Platform Devices and Drivers," *Driver Implementer's API Guide*. [Online]. Available: https://docs.kernel.org/driver-api/driver-model/platform.html. [Accessed: Oct. 16, 2026].

[2] G. Likely, "Linux and the Devicetree," *The Linux Kernel Documentation*. [Online]. Available: https://docs.kernel.org/devicetree/usage-model.html. [Accessed: Oct. 16, 2026].

[3] The Linux Kernel Documentation, "Pulse Width Modulation (PWM) interface," *Driver Implementer's API Guide*. [Online]. Available: https://docs.kernel.org/driver-api/pwm.html. [Accessed: Oct. 16, 2026].

[4] The Linux Kernel Documentation, "Real Time Clock (RTC) Drivers for Linux," *The Linux Kernel User's and Administrator's Guide*. [Online]. Available: https://docs.kernel.org/admin-guide/rtc.html. [Accessed: Oct. 16, 2026].

[5] Broadcom Corporation, *BCM2835 ARM Peripherals*, Cambridge, U.K.: Broadcom Europe Ltd., Feb. 2012. [Online]. Available: https://datasheets.raspberrypi.com/bcm2835/bcm2835-peripherals.pdf. [Accessed: Oct. 16, 2026].

[6] Maxim Integrated (Analog Devices), "DS1307 64 x 8, Serial, I2C Real-Time Clock," Datasheet, Rev. 5, Mar. 2015. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/DS1307.pdf. [Accessed: Oct. 16, 2026].

[7] Raspberry Pi Ltd, "Device Trees, overlays, and parameters," in *Raspberry Pi Documentation: Configuration*. [Online]. Available: https://www.raspberrypi.com/documentation/computers/configuration.html#device-trees-overlays-and-parameters. [Accessed: Oct. 16, 2026].

[8] Raspberry Pi Ltd, "The Linux kernel," in *Raspberry Pi Documentation*. [Online]. Available: https://www.raspberrypi.com/documentation/computers/linux_kernel.html. [Accessed: Oct. 16, 2026].

[9] J. Corbet, A. Rubini, and G. Kroah-Hartman, *Linux Device Drivers*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005. [Online]. Available: https://lwn.net/Kernel/LDD3/. [Accessed: Oct. 16, 2026].